Foundations of Deep Learning

Lecture 06

Transformers

Aurelien Lucchi

Fall 2026

Transformers in Deep Learning

- Transformers are a type of deep learning architecture introduced in the paper "Attention is All You Need" by Vaswani et al. (2017).
- They have become the **state-of-the-art** approach in many natural language processing (NLP) tasks, such as machine translation and language understanding.

Applications of Transformer Architectures

- Transformers have found success in a wide range of applications beyond NLP (Natural Language Processing), including:
  - Machine Translation
  - Image Generation (e.g., DALL-E)
  - Speech Recognition
  - Protein Folding
- They excel in tasks requiring sequence-to-sequence modeling.

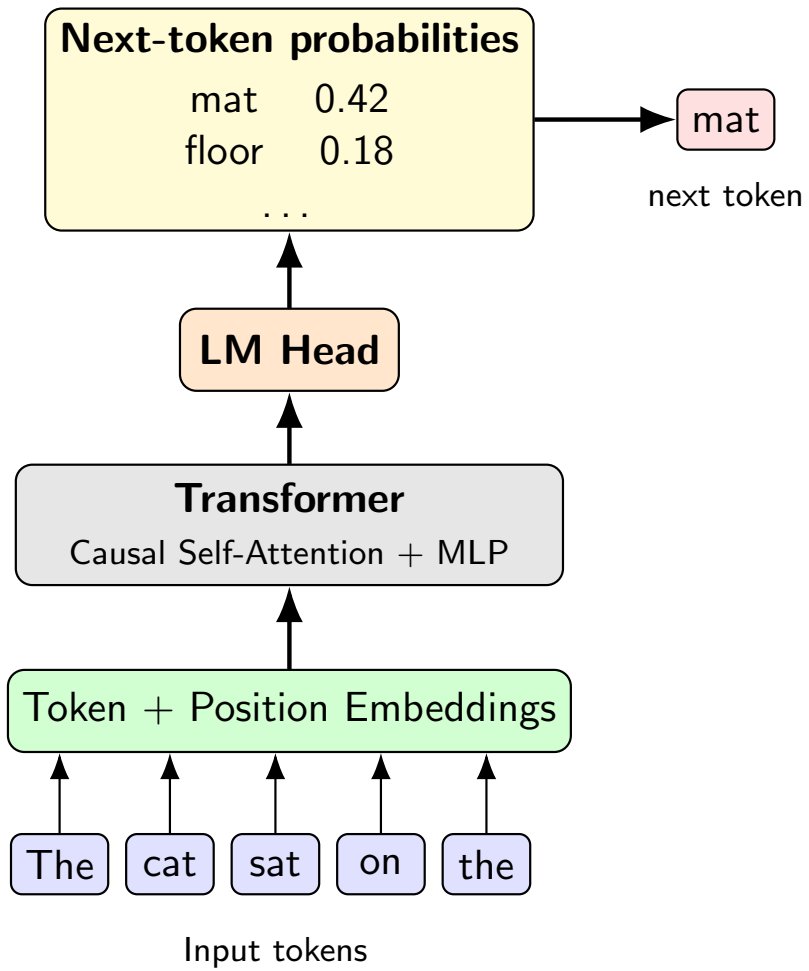


Figure: Illustration of a Transformer for autoregressive language modeling.

We will focus on text modeling, but Transformers are also widely used in other domains, such as computer vision.

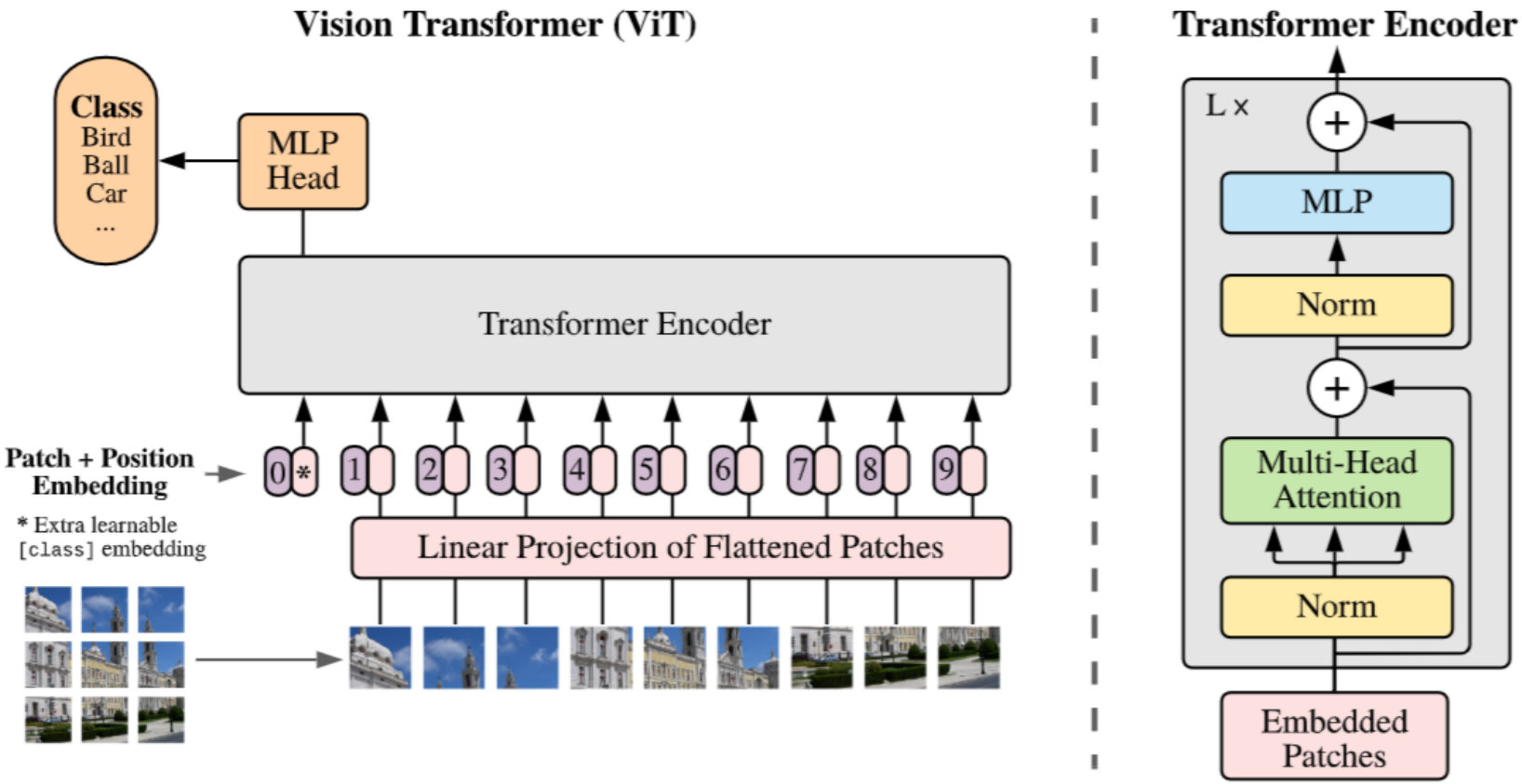


Figure: Vision Transformers (Dosovitskiy et al. 2021)

Transformer Model

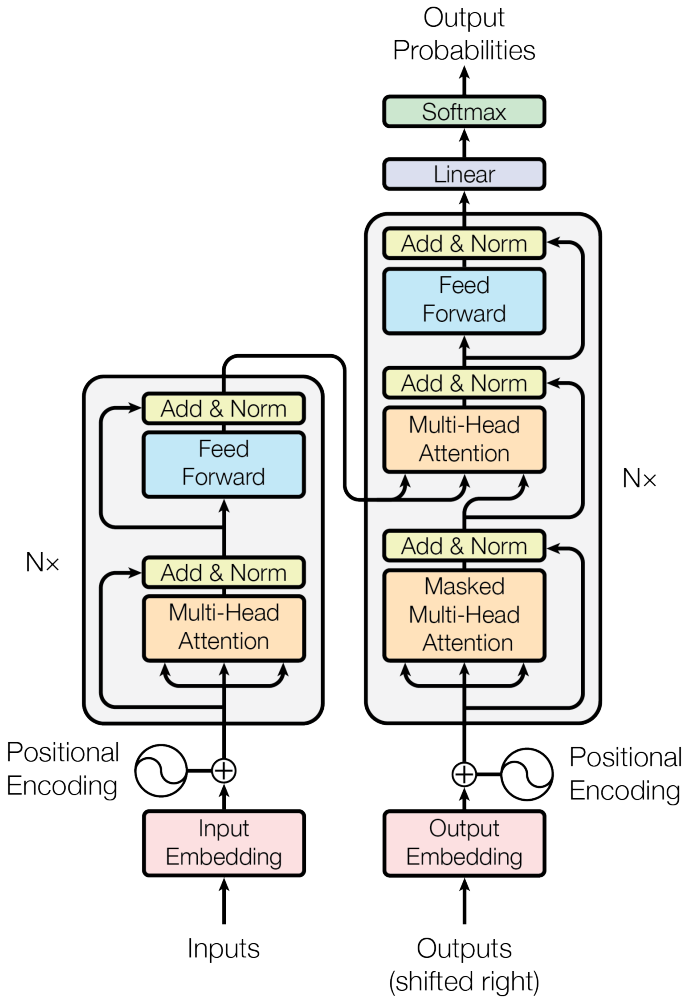


Figure: A standard transformer architecture, showing on the left an encoder, and on the right a decoder. Source: Attention Is All You Need, Vaswani et al.

Transformer Model (simplified)

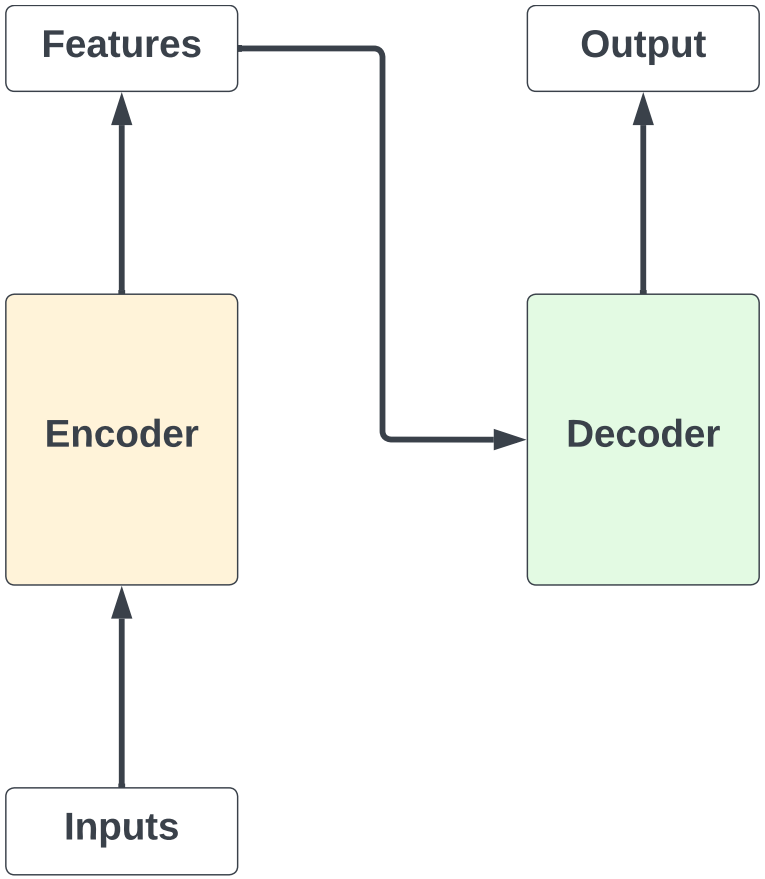


Figure: Simplified standard transformer architecture: The encoder extracts features from an input sentence, and the decoder uses the features to produce an output sentence (e.g. for a translation task). The encoder is not necessary for all tasks.

Today's program

- Introduce a simple decoder-only Transformer architecture
- Analyze each architectural component in detail
- Provide a PyTorch implementation to illustrate the concepts

Section 1

Language Modelling

Autoregressive Models

- A language model assigns probabilities to sequences of tokens:

$$p(x_1, \ldots, x_L)$$

$$P(\text{the, cat, chased, the, ball}) = 0.03$$

$$P(\text{cat, the, chased, ball, the}) = 0.0002 \qquad \text{(captures syntax)}$$

$$P(\text{the, ball, chased, the, cat}) = 0.002 \qquad \text{(captures meaning)}$$

- Language models can generate text by sampling:

$$x_{1:L} \sim p(x_1, \ldots, x_L)$$

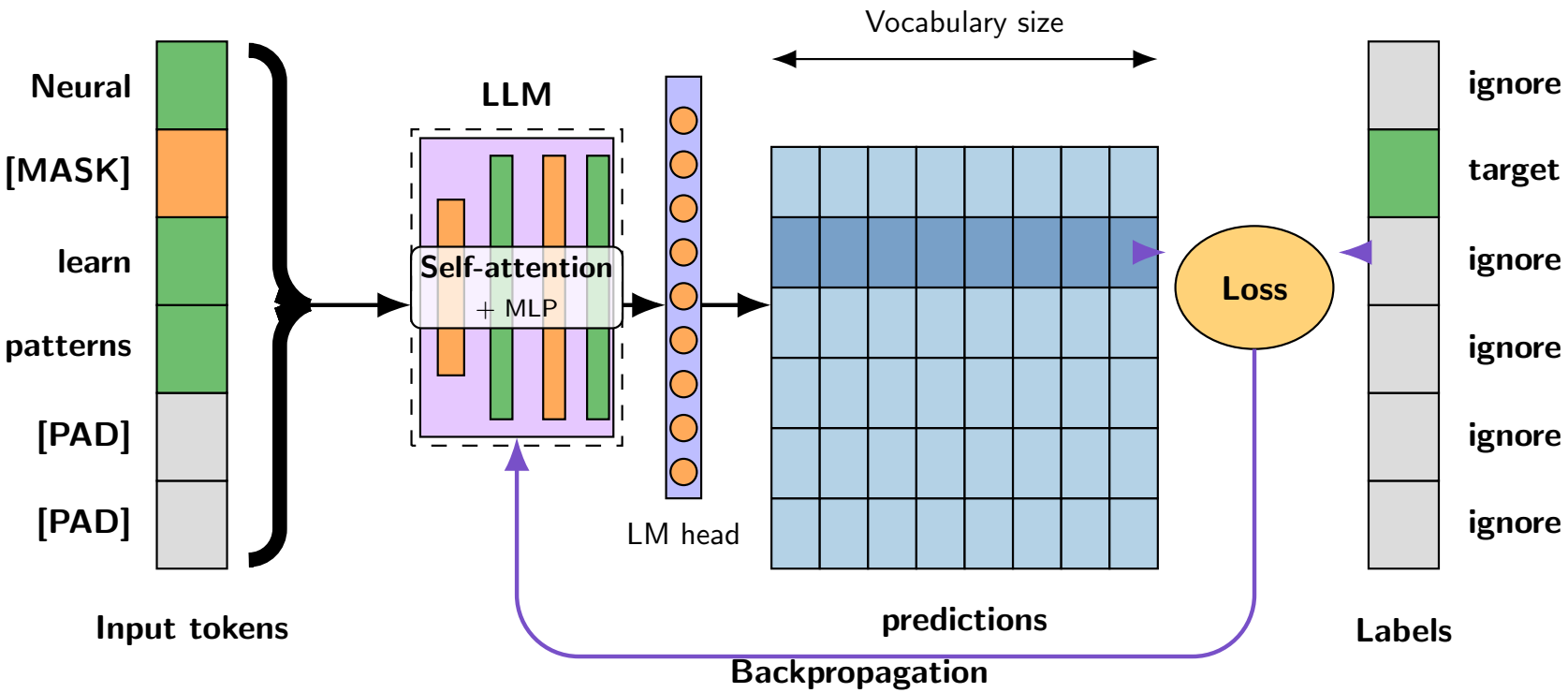
- **Autoregressive formulation:**

$$p(x_1, \ldots, x_L) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_{1:2}) \cdots = \prod_{i=1}^{L} p(x_i \mid x_{<i})$$

Exact factorization via the chain rule

$\Rightarrow$ Training reduces to predicting the next token from previous ones.

Typical Pipeline



Tokenization = process of converting raw text into a sequence of discrete units (tokens) that can be processed by LLM.

Key Ideas:

- Tokens can be characters, subwords, or full words.
- Subword tokenization $\implies$ balance flexibility and efficiency.
- Reduces vocabulary size while preserving semantic granularity.

Pytorch: `nn.Embedding`(V, d) is basically a lookup table:

$$E \in \mathbb{R}^{V \times d}, \qquad V = \text{vocab_size}, \qquad d = n_{\text{embd}}$$

Each row corresponds to one token.

Tokenization example using GPT2

Token	ID	L2 Norm	Embedding Vector
Like	7594	3.30	$[-0.166, -0.080, 0.086, \ldots]$
a	257	2.57	$[-0.051, 0.006, 0.047, \ldots]$
fish	5916	3.26	$[0.027, 0.100, 0.066, \ldots]$
out	503	2.71	$[-0.048, -0.044, 0.034, \ldots]$
of	286	2.59	$[-0.057, 0.018, 0.033, \ldots]$
water	1660	3.05	$[0.065, 0.020, 0.023, \ldots]$

Cross-Entropy Loss (one more time)

Goal. In multi-class classification, the model first produces **logits** $o_1, \ldots, o_C$, one for each class.

These logits are transformed into probabilities with the softmax:

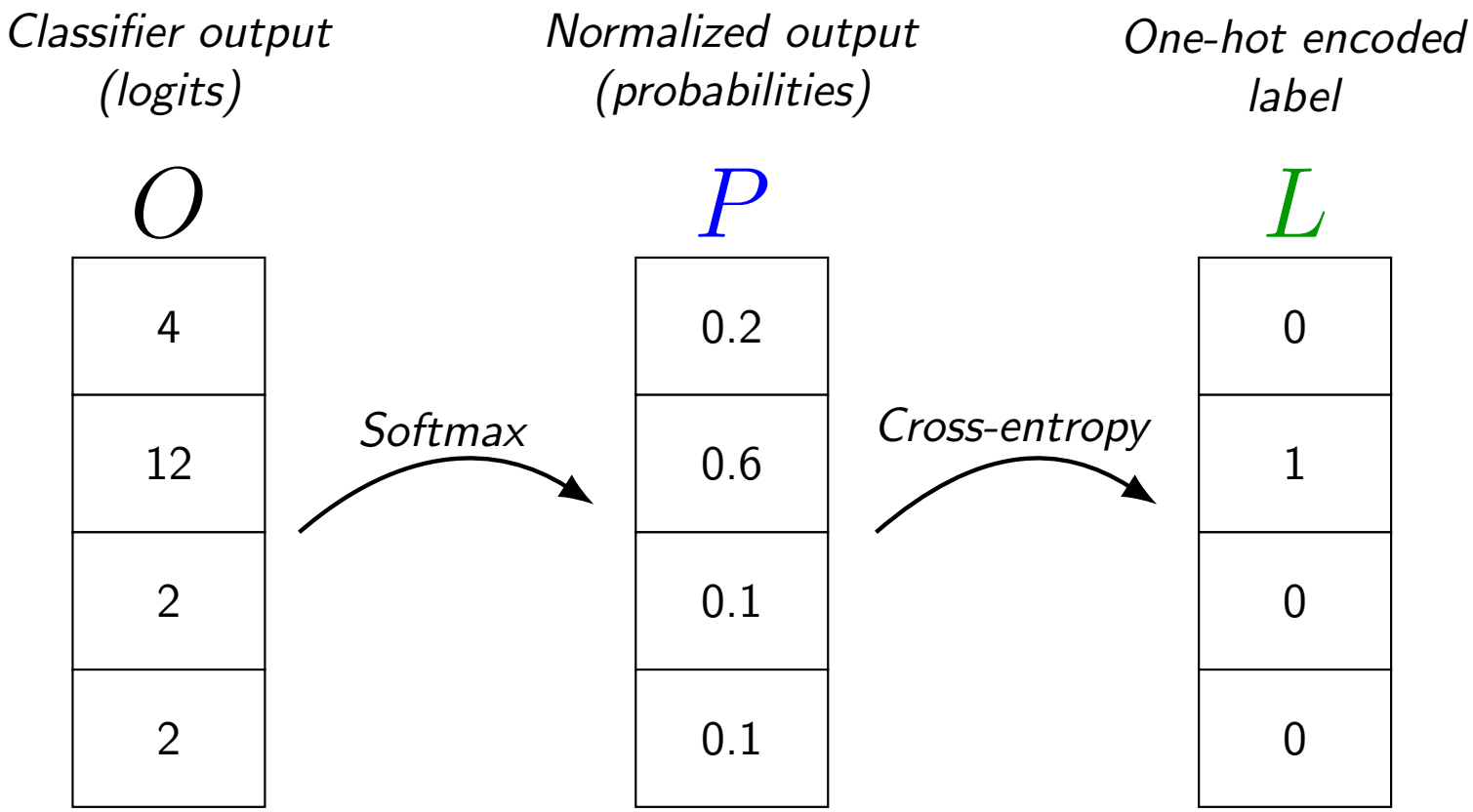
$$p_i = \frac{e^{o_i}}{\sum_j e^{o_j}}.$$

The true label is represented by a one-hot vector y, and the cross-entropy loss compares the predicted probabilities to that target:

$$\mathcal{L}_{\mathrm{CE}}(p, y) = -\sum_{i=1}^{C} y_i \log p_i.$$

If the correct class receives high probability, the loss is small. If the correct class receives low probability, the loss is large.

Cross-Entropy: Example



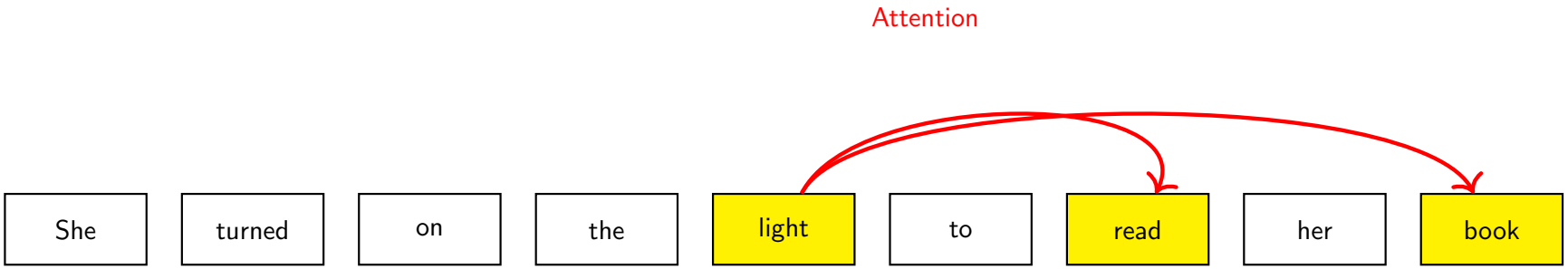
$$\text{CrossEntropy}(P, L) = -\sum_i L_i \log(P_i)$$

Section 2

Attention Models

Transformer Model: Attention Mechanism

- Unlike traditional recurrent neural networks (RNNs) or convolutional neural networks (CNNs), Transformers rely on a **self-attention mechanism** to process input data.
- Self-attention allows Transformers to capture **dependencies** between different words in a sentence.

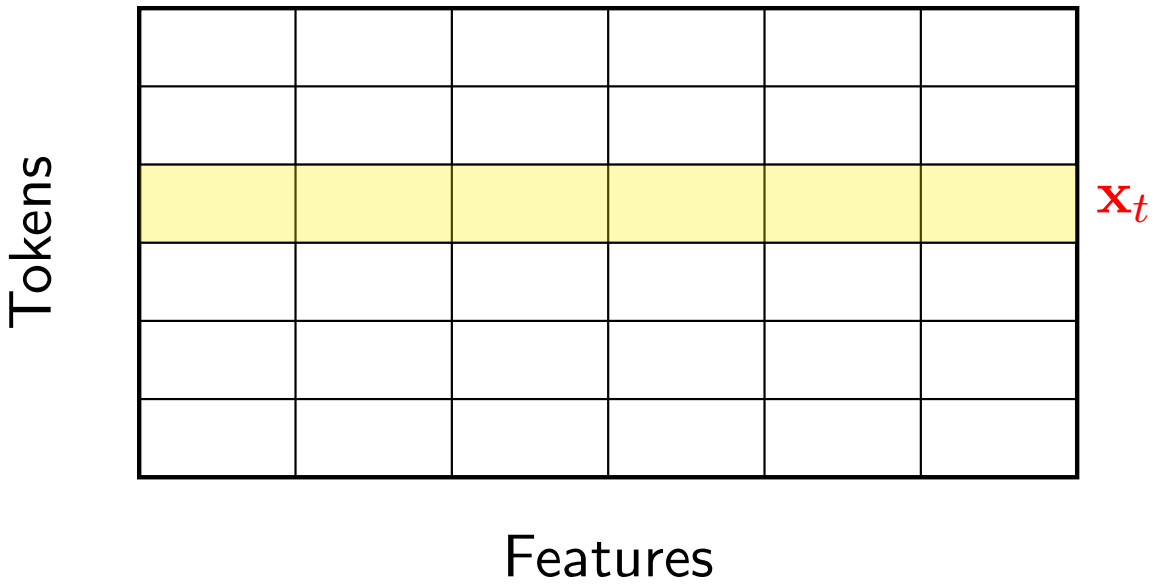


Transformer Model: Input data

Input data to a transformer is a set of vectors $\{\mathbf{x}_t\}$ of dimensionality d, where $n = 1, \ldots, T$. We refer to these data vectors as **tokens**, where a token might, e.g., correspond to:

- A word within a sentence
- A patch within an image
- An amino acid within a protein

The elements x_{ti} of the tokens are called **features**.



- **Goal:** map input tokens $\mathbf{x}_1, \ldots, \mathbf{x}_T$ to another set $\mathbf{y}_1, \ldots, \mathbf{y}_T$ that captures a richer semantic structure.
- With attention, this dependence should be stronger for those inputs $\mathbf{x}_m$ that are particularly important for determining the modified representation of $\mathbf{y}_t$.

Mathematically:

$$\mathbf{y}_t = \sum_{m=1}^{T} a_{tm}\mathbf{x}_m, \qquad a_{tm} = \frac{\exp(\mathbf{x}_t^\top \mathbf{x}_m)}{\sum_{k=1}^{T} \exp(\mathbf{x}_t^\top \mathbf{x}_k)}$$

where a_{tm} are the attention coefficients.

$$\rightsquigarrow \text{Matrix form: } \mathbf{Y} = \text{Softmax}(\mathbf{X}\mathbf{X}^\top)\mathbf{X}$$

Softmax(A) applies the exponential function to each element of a matrix A and then normalizes each row so that the sum of the elements in each row equals one.

Attention Mechanism

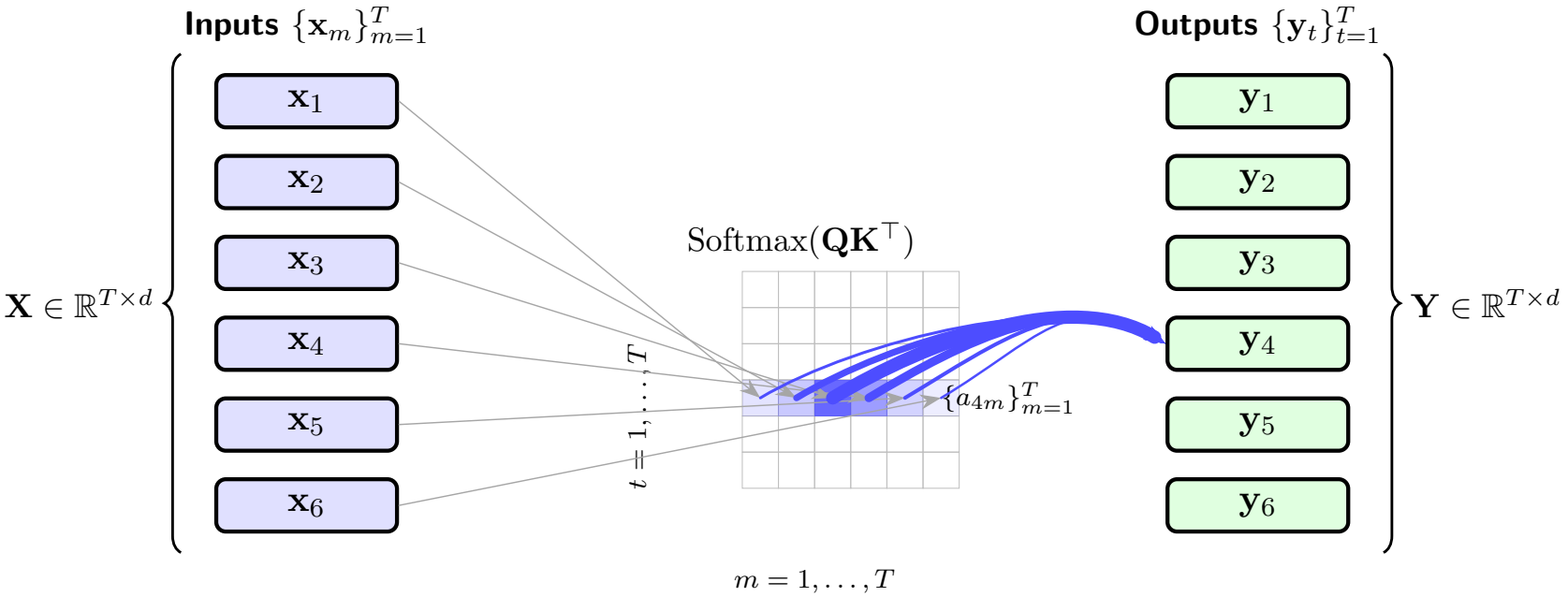


Figure: Self-attention computes each output token $\mathbf{y}_t$ as a weighted combination of input tokens, using the corresponding row of the attention matrix.

Attention Mechanism: Adjustable Parameters

We redefine the modified feature vectors as the result of applying a linear transformation to the original vectors, expressed as $\mathbf{XU}$, where $\mathbf{U}$ is a $d \times d$ matrix containing learnable weight parameters.

$$\rightsquigarrow \text{Matrix form: } \mathbf{Y} = \text{Softmax}(\mathbf{XUU}^\top\mathbf{X}^\top)\mathbf{XU}$$

Add more flexibility by defining 3 matrices (query, key, value):

$$\mathbf{Q} = \mathbf{XW}^q \quad \mathbf{K} = \mathbf{XW}^k \quad \mathbf{V} = \mathbf{XW}^v. \tag{1}$$

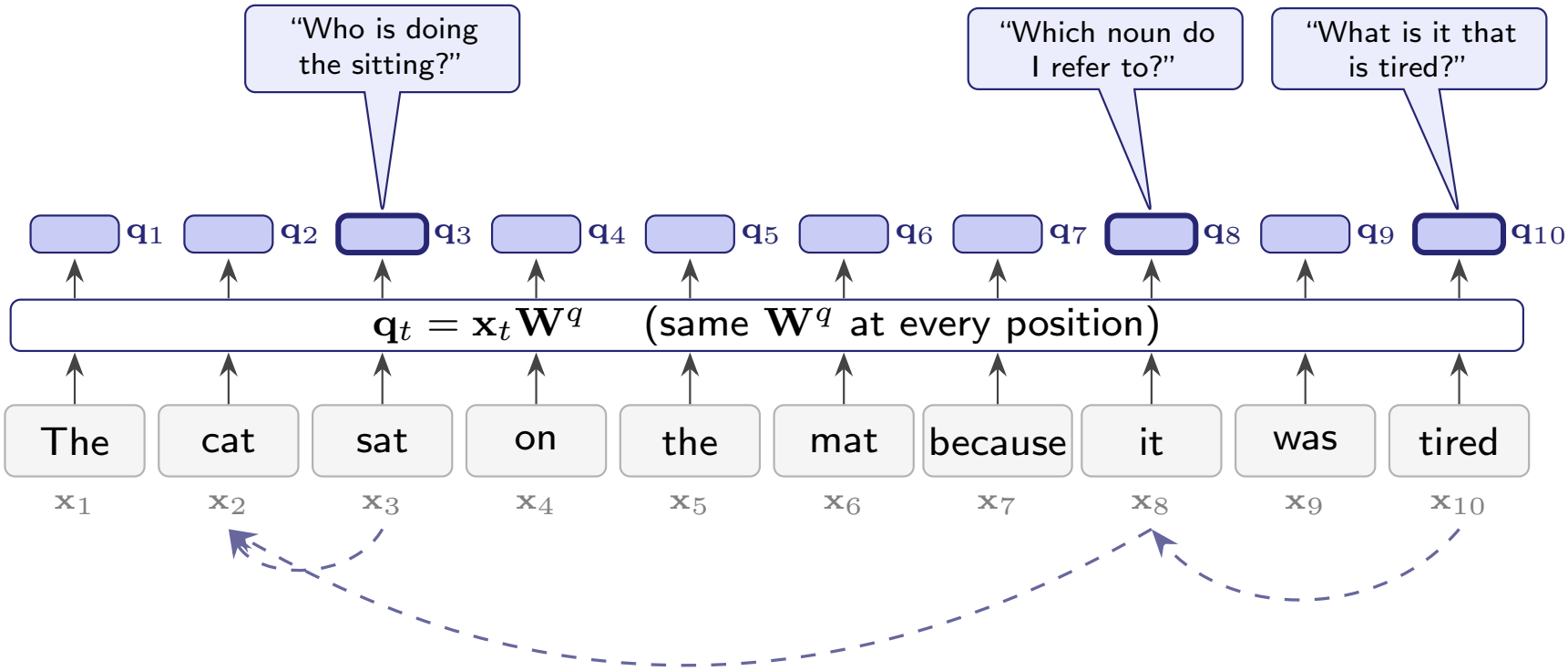
$$\rightsquigarrow \text{Matrix form: } \mathbf{Y} = \text{Softmax}(\mathbf{QK}^\top)\mathbf{V}$$

Note on nomenclature

The names queries, keys and values come from the field of information retrieval systems:

- The **query** is the question or request you're asking.
  - What am I looking for?
- The **keys** are indexed labels or features that allow the system to understand where the relevant information is.
  - What do I contain?
- The **values** are the actual content or information retrieved once a match is made.

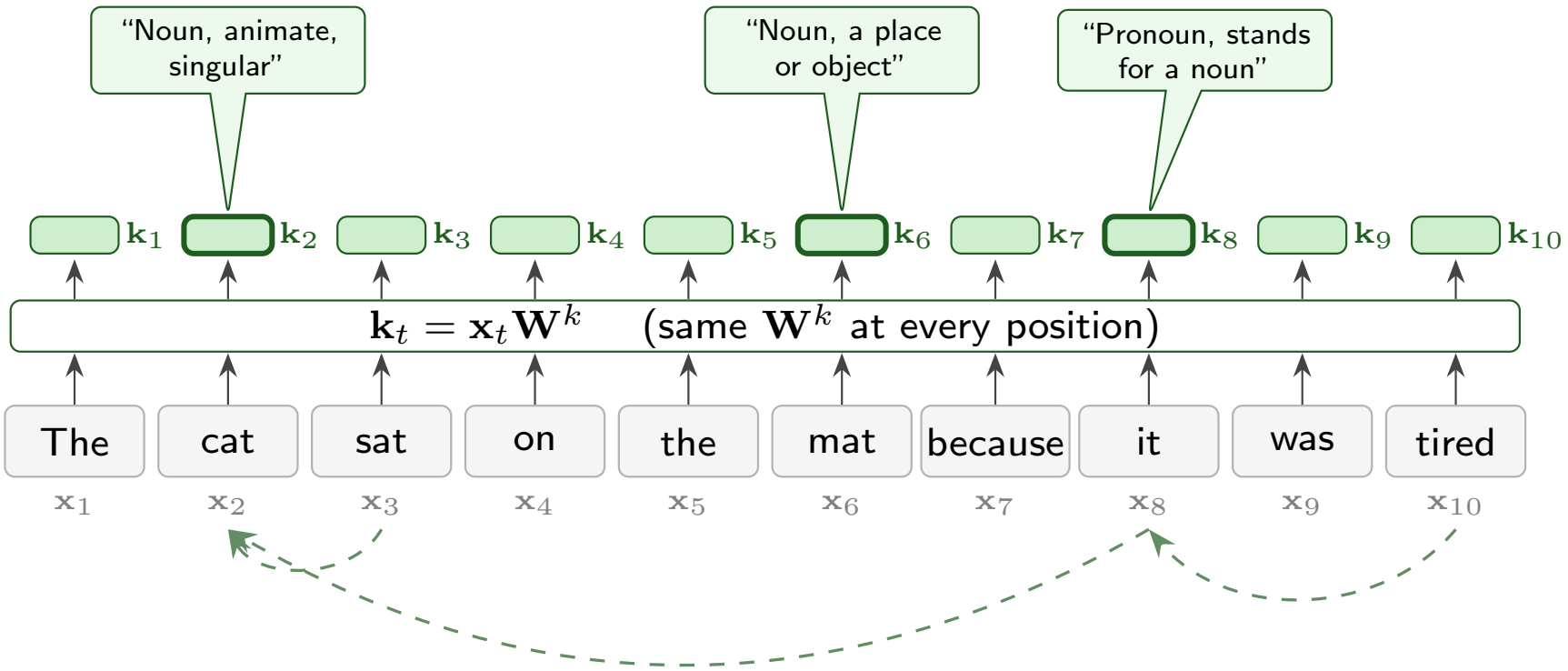
Queries: what is each token looking for?



answers are found among **earlier** tokens, by matching $\mathbf{q}_t$ against their **keys** $\mathbf{k}_m$

A query is a learned summary of **what token t needs** from the context — not of what it contains.

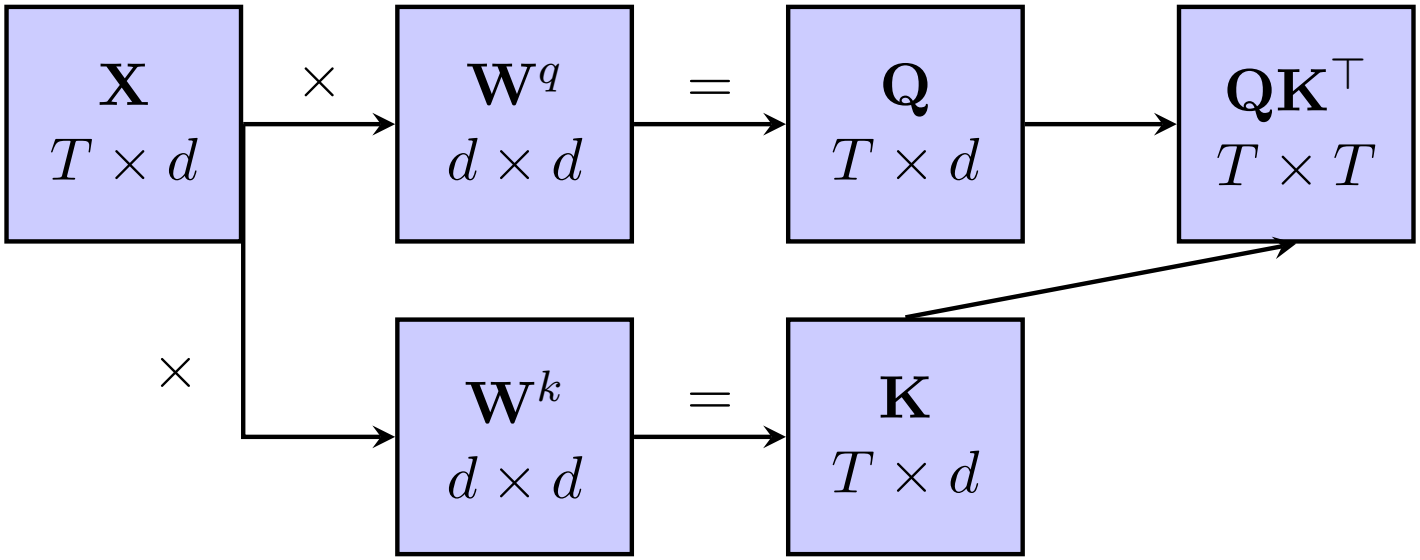
Keys: what does each token advertise?



a later query $\mathbf{q}_t$ finds token m when $\mathbf{q}_t^\top \mathbf{k}_m$ is large (e.g. "it" $\rightarrow$ "cat")

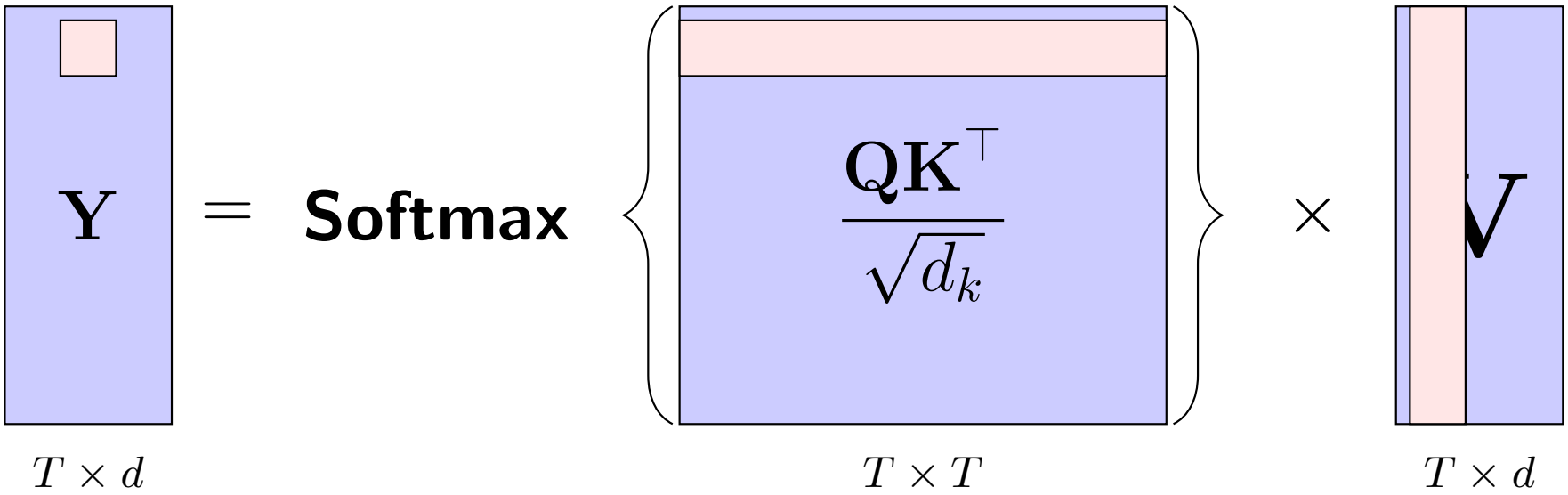
A key is a learned label of **what token m offers** — it is used only for matching, not passed on.

Attention Mechanism: Adjustable Parameters



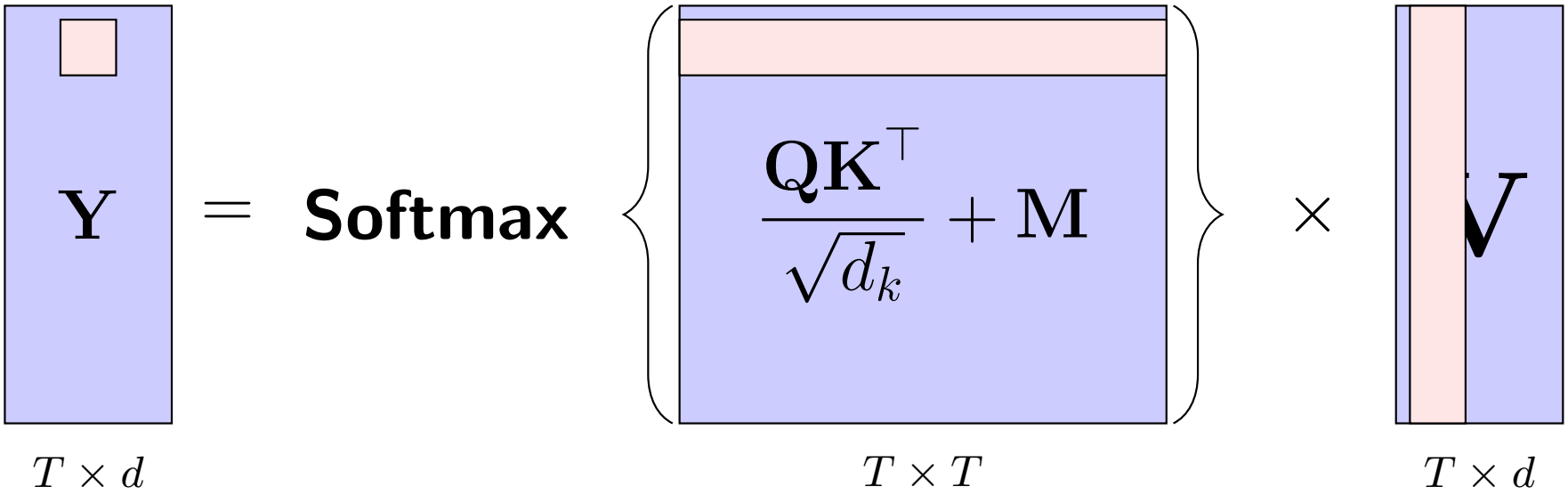
- Matrices $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_v \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d_v \times d_v}$ are learnable parameters, and each layer is initialized with an independent set of weights
- Figure assumes $d_v = d_k = d$ for simplicity

Scaling



- **Scaling**: The dot product sums over d_k components, so its variance grows proportionally to $d_k \rightsquigarrow$ divide by $\sqrt{d_k}$ to get constant variance.

Masking



- **Masking:** Prevent tokens from attending to future positions during training.

Unmasked Attention:

$$\tilde{\mathbf{A}} = \frac{1}{\sqrt{d_k}}\mathbf{Q}\mathbf{K}^\top, \quad \mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K$$

Causal Mask:

$$\mathbf{M}_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Masked Attention:

$$\mathbf{A} = \text{softmax}(\tilde{\mathbf{A}} + \mathbf{M})$$

Effect: Enforces autoregressive generation by hiding future tokens.

Softmax turns $-\infty$ into zero weight. The masked attention weights are:

$$\mathbf{A} = \mathrm{softmax}(\tilde{\mathbf{A}} + \mathbf{M}).$$

Row-wise, this is:

$$\mathbf{A}_{ij} = \frac{\exp\big((\tilde{\mathbf{A}} + \mathbf{M})_{ij}\big)}{\sum_{\ell=1}^{T} \exp\big((\tilde{\mathbf{A}} + \mathbf{M})_{i\ell}\big)}.$$

Now if $j > i$, then $(\tilde{\mathbf{A}} + \mathbf{M})_{ij} = -\infty$, so:

$$\exp(-\infty) = 0 \quad \Longrightarrow \quad \mathbf{A}_{ij} = 0 \qquad \text{for all } j > i.$$

That is the masking effect: future attention weights become exactly zero.

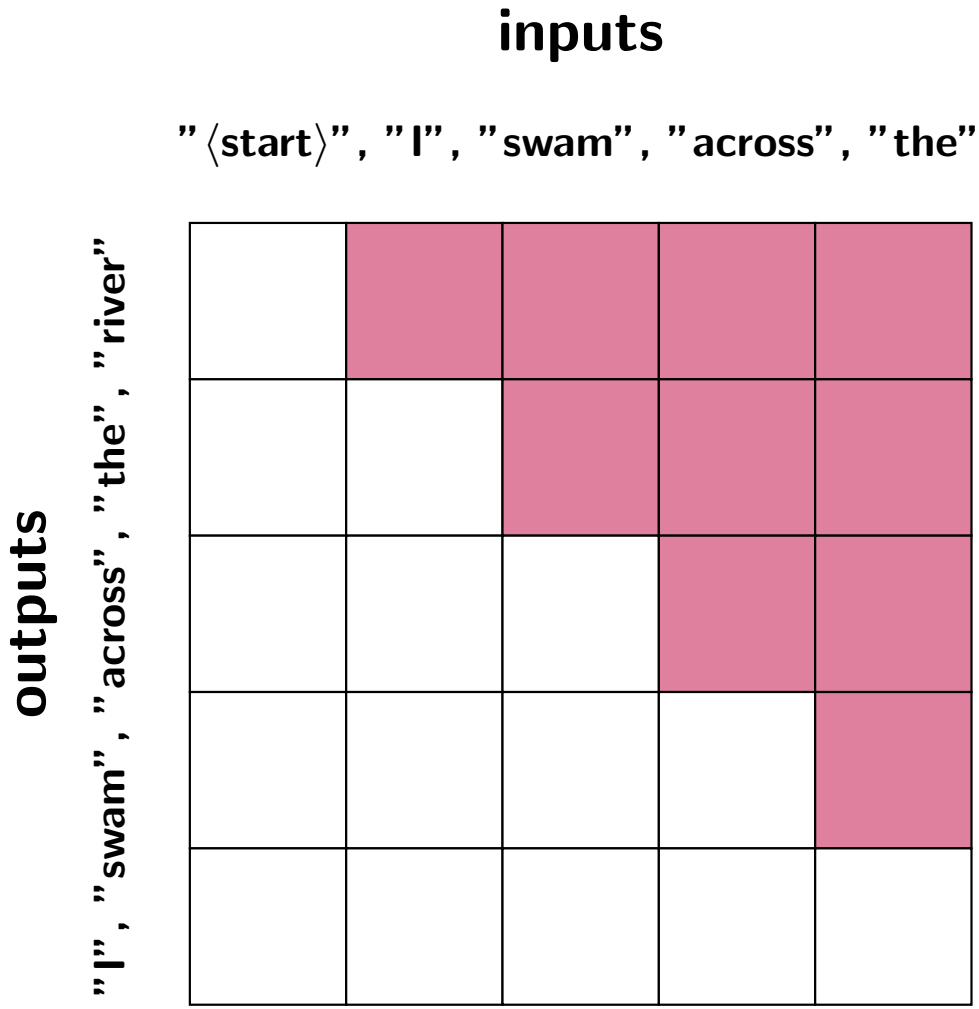


Figure: Illustration of the mask matrix where attention weights for the purple entries are set to zero. Note the addition of a start token at the beginning of the input to ensure a consistent alignment between input and output tokens.

Why this guarantees causal outputs. The attention output is computed using values

$$\mathbf{V} = \mathbf{X}\mathbf{W}^V, \qquad \mathbf{Y} = \mathbf{A}\mathbf{V}.$$

Component-wise, this can be written as

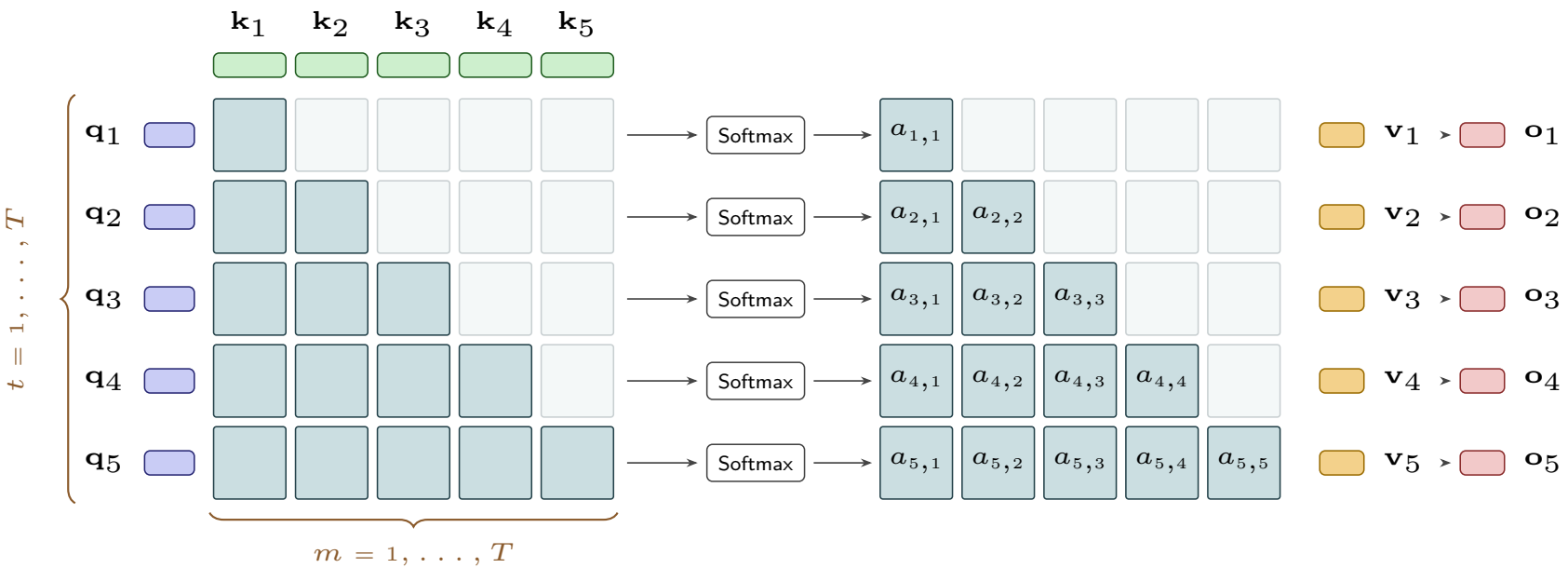
$$\mathbf{Y}_i = \sum_{j=1}^{T} \mathbf{A}_{ij}\mathbf{V}_j.$$

But since $\mathbf{A}_{ij} = 0$ for all $j > i$, this reduces to

$$\mathbf{Y}_i = \sum_{j \le i} \mathbf{A}_{ij}\mathbf{V}_j.$$

So token i's representation depends only on the present/past, never the future.

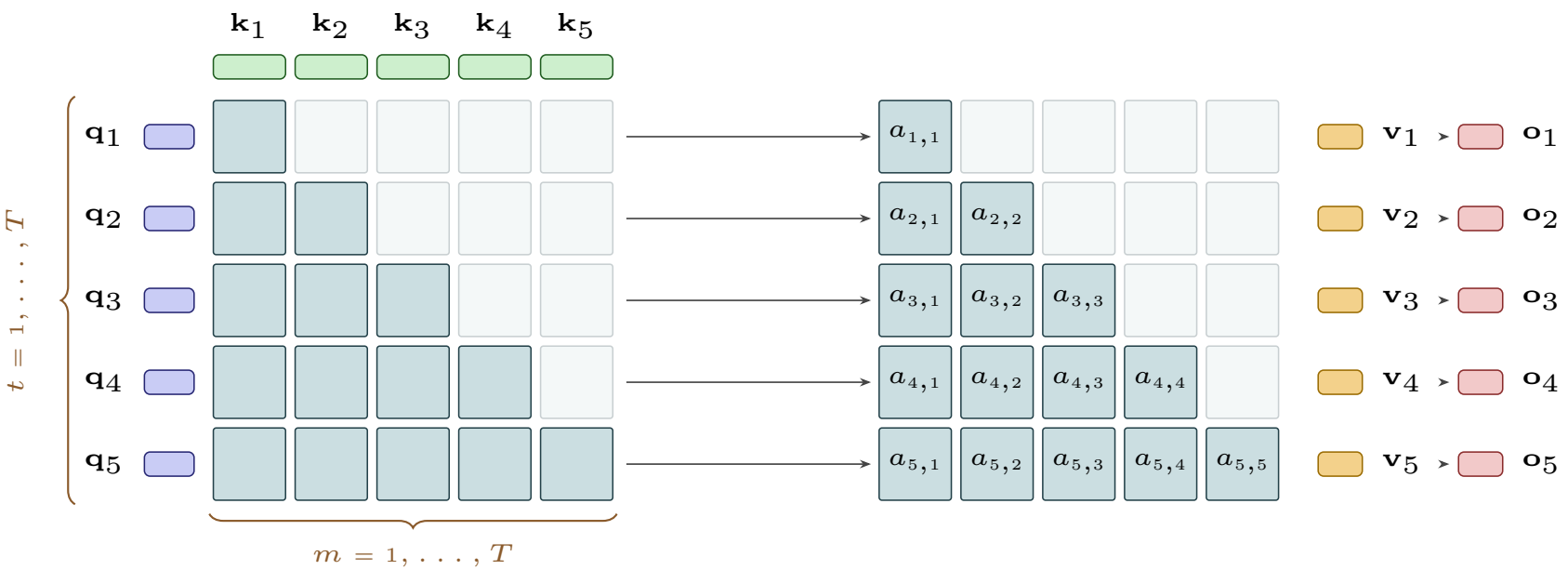
Attention Matrix Softmax($\mathbf{QK}^\top$)



$$\mathbf{o}_t = \sum_{i=1}^{t} \frac{\exp\left(\mathbf{q}_t \mathbf{k}_i^\top\right)}{\sum_{j=1}^{t} \exp\left(\mathbf{q}_t \mathbf{k}_j^\top\right)} \mathbf{v}_i$$

- Each output $\mathbf{o}_t$ needs a score against every $\mathbf{k}_i$, $i \le t$: $\sum_{t=1}^{T} t = T(T+1)/2$ query–key pairs.
- The total cost is therefore $\mathcal{O}(T^2 d)$.

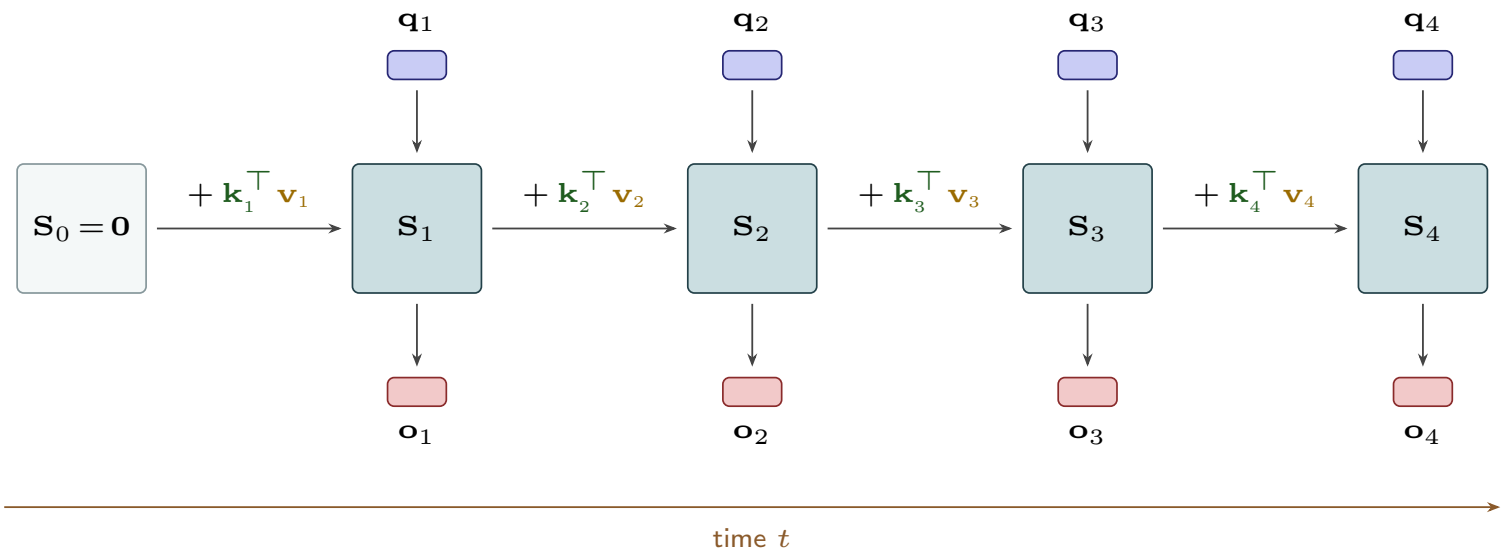
Attention Matrix - Linear case



$$\mathbf{o}_t = \sum_{i=1}^{t} \mathbf{q}_t \mathbf{k}_i^\top \mathbf{v}_i = \mathbf{q}_t \underbrace{\sum_{i=1}^{t} \mathbf{k}_i^\top \mathbf{v}_i}_{\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}} \qquad \mathbf{S}_{t+1} = \mathbf{S}_t + \mathbf{k}_t \mathbf{v}_t$$

Attention Matrix - Linear Case

- $\mathbf{S}_t \in \mathbb{R}^{d \times d}$ is a **fixed-size** summary of $\mathbf{k}_1, \ldots, \mathbf{k}_t$ and $\mathbf{v}_1, \ldots, \mathbf{v}_t$.
- Each token contributes a rank-one update $\mathbf{k}_t^\top \mathbf{v}_t$; the state never grows with t.
- The output is a linear read-out of the current state, $\mathbf{o}_t = \mathbf{q}_t \mathbf{S}_t$.

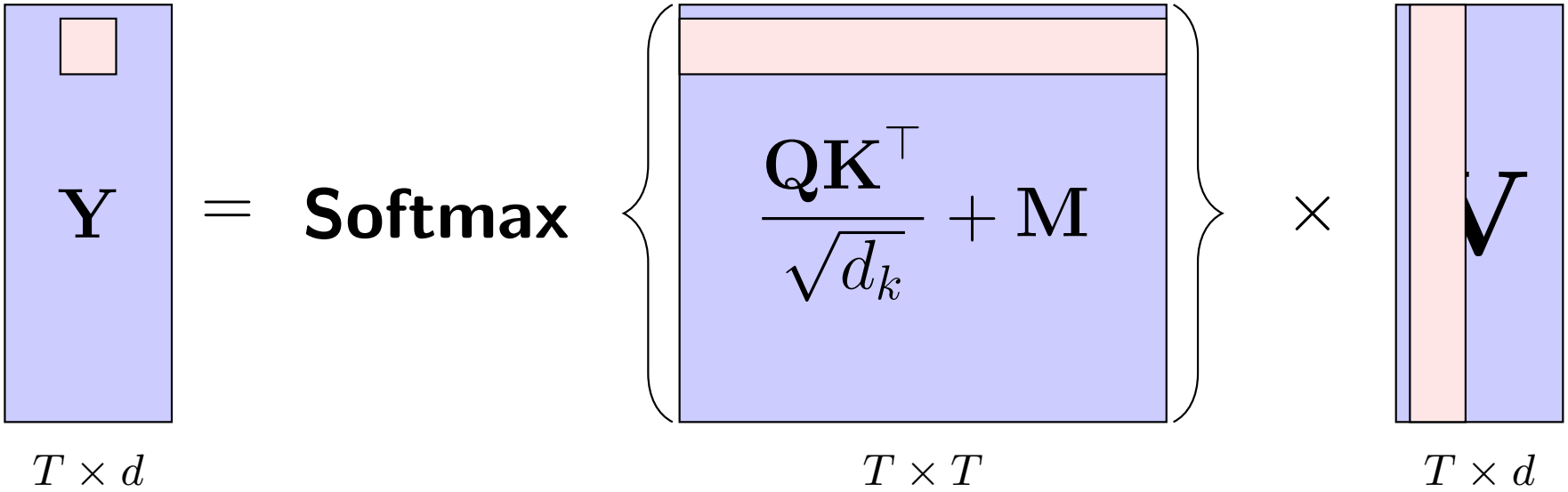


Decay and Gating in Linear Attention

$$\mathbf{S}_t \;=\; \mathbf{G}_t\,\mathbf{S}_{t-1} \;+\; \mathbf{k}_t^\top \mathbf{v}_t, \qquad \mathbf{o}_t \;=\; \mathbf{q}_t\,\mathbf{S}_t$$

- **Linear attention.** $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$
  $\mathbf{G}_t = \mathbf{I}$: every token is written with equal weight and never removed, so the state saturates as t grows.
- **Fixed decay (RetNet).** $\mathbf{S}_t = \gamma\,\mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \quad \gamma \in (0,1)$
  Contributions age geometrically: $\mathbf{k}_i^\top \mathbf{v}_i$ carries weight $\gamma^{\,t-i}$. One scalar per head, fixed after training — the model cannot choose what to forget.
- **Data-dependent decay (Mamba-2).** $\mathbf{S}_t = \gamma_t\,\mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t,\ \gamma_t = f(\mathbf{x}_t)$
  The decay now depends on the current token: the model can hold the state ($\gamma_t \to 1$) or flush it ($\gamma_t \to 0$) in a content-dependent way.
- **DeltaNet.** $\mathbf{S}_t = \big(\mathbf{I} - \beta_t \mathbf{k}_t^\top \mathbf{k}_t\big)\mathbf{S}_{t-1} + \beta_t \mathbf{k}_t^\top \mathbf{v}_t$
  DeltaNet writes only the prediction residual, allowing it to correct existing key–value associations instead of repeatedly accumulating them.
- **Gated linear attention (GLA).** $\mathbf{S}_t = \mathrm{Diag}(\boldsymbol{\alpha}_t)\,\mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t,$
  $\boldsymbol{\alpha}_t = f(\mathbf{x}_t) \in (0,1)^d$
  One gate **per key channel** instead of one per head: different parts of the state decay at different rates, giving multiple effective timescales.

Attention Mechanism vs CNN



- In contrast to a conventional neural network, where activations are multiplied by fixed weights, the signal paths in this context involve multiplicative interactions between activation values.
- Instead of static weights, the activations here are dynamically multiplied by data-dependent attention coefficients.

Head: Attention layer described so far enables the output vectors to focus on input-specific patterns and is referred to as an **attention head**

$\rightsquigarrow$ there might be multiple patterns of attention that are relevant at the same time.

Concatenation: Concatenate outputs from multiple attention heads and apply a linear transformation to generate the final output.

Let head_i denote the output of the i-th head. Then, the multi-head attention output is:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)\mathbf{W}^O$$

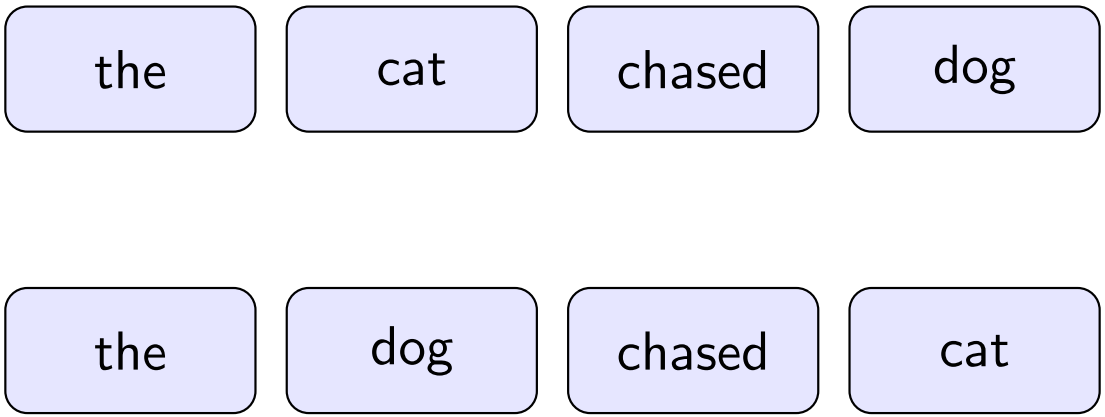
Here, $\mathbf{W}^O \in \mathbb{R}^{hd \times d_{\text{model}}}$ is the output projection matrix.

Section 3

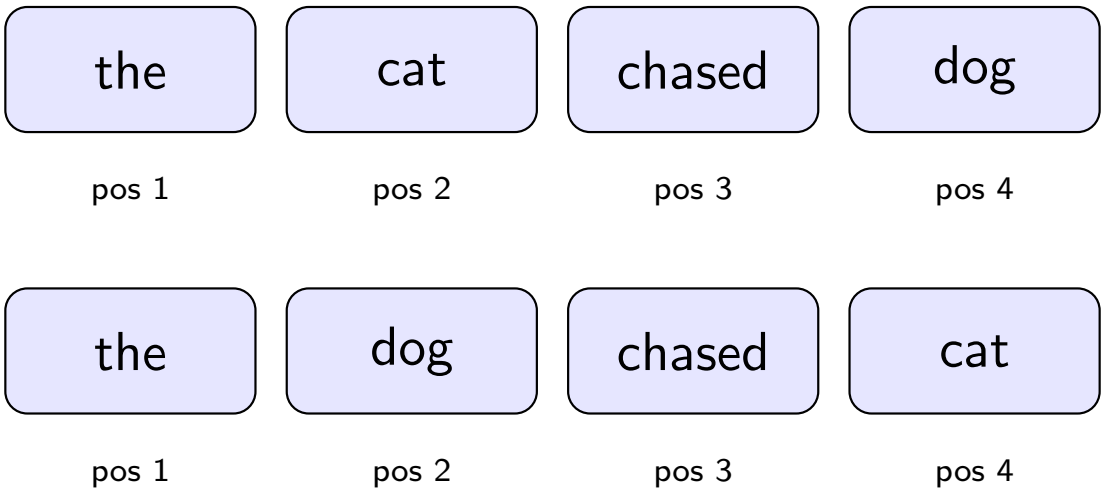
Positional Embeddings

Why do we need positional embeddings?

Transformers process tokens in parallel, so without positional information they do not know the difference between:



Why do we need positional embeddings?



Standard idea:

$$\mathbf{x}_t \leftarrow \mathbf{x}_t + \mathbf{p}_t$$

where $\mathbf{x}_t$ is the token embedding and $\mathbf{p}_t$ is a positional embedding.

RoPE: RoPE uses a different idea:

$$\mathbf{q}_t \mapsto \mathbf{R}_t \mathbf{q}_t, \qquad \mathbf{k}_t \mapsto \mathbf{R}_t \mathbf{k}_t$$

Instead of adding position vectors, RoPE rotates queries and keys depending on the token position.

Core idea: rotate each 2D pair

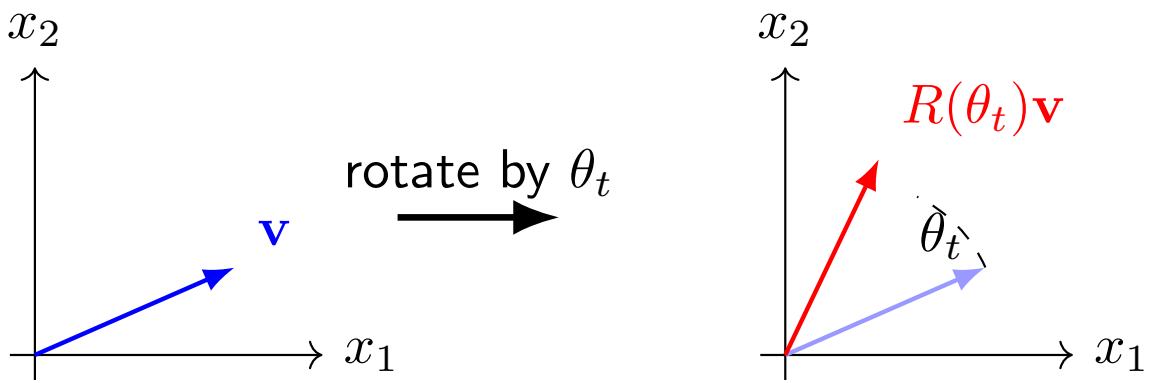
RoPE groups embedding coordinates into pairs:

$$(x_{t,1}, x_{t,2}),\ (x_{t,3}, x_{t,4}),\ \ldots$$

and rotates each pair by an angle depending on the position t.

$$\mathbf{R}(\theta_t) = \begin{pmatrix} \cos\theta_t & -\sin\theta_t \\ \sin\theta_t & \cos\theta_t \end{pmatrix} \Rightarrow \begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = \mathbf{R}(\theta_t) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

(dropped subscript t to avoid clutter)



So position is encoded as a rotation angle in embedding space.

Why does RoPE capture relative position?

Attention uses dot products between queries and keys:

$$\text{score}(i, j) = \mathbf{q}_i^\top \mathbf{k}_j$$

With RoPE:

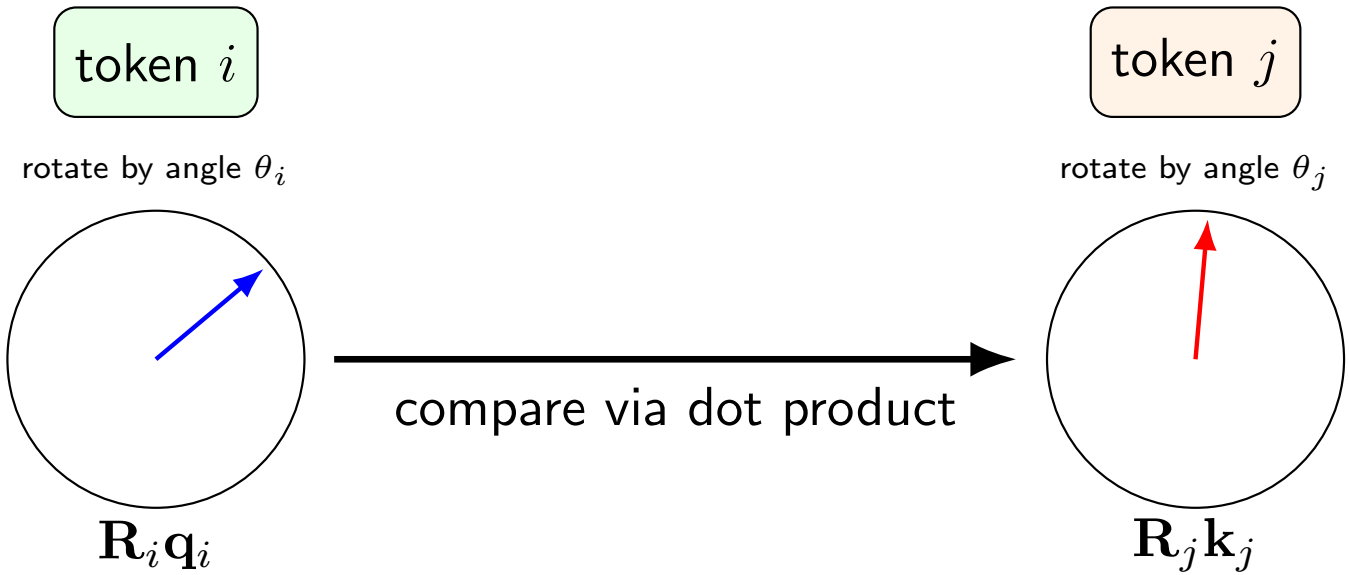
$$\text{score}(i, j) = (\mathbf{R}_i \mathbf{q}_i)^\top (\mathbf{R}_j \mathbf{k}_j)$$

A key property of rotations is that this depends on the relative angle, hence on the relative position $i - j$.

$$(\mathbf{R}_i \mathbf{q})^\top (\mathbf{R}_j \mathbf{k}) \quad \text{depends on} \quad \mathbf{R}_{j-i}$$

Why? Use two basic facts about rotations: $R_\theta^\top = R_{-\theta}$ and $R_\alpha R_\beta = R_{\alpha+\beta}$.

Why does RoPE capture relative position?



What matters is the angle difference $\theta_j - \theta_i \implies$ This is why RoPE naturally gives the model access to relative position information.

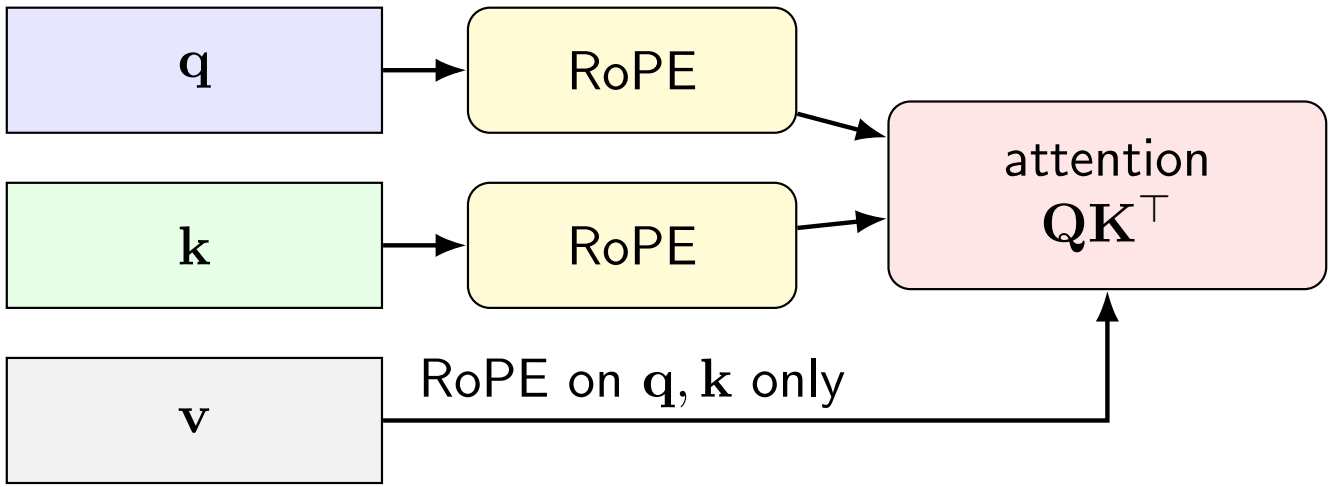
How it is implemented in practice

For each head, RoPE is applied to $\mathbf{q}$ and $\mathbf{k}$, usually not to $\mathbf{v}$.

For coordinate pairs (x_{2m}, x_{2m+1}):

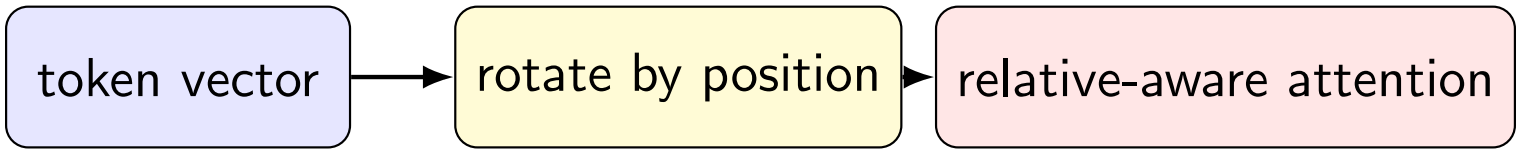
$$\theta_{t,m} = t\,\omega_m \qquad \text{with} \qquad \omega_m = 10000^{-2m/d}$$

$$\begin{pmatrix} x'_{2m} \\ x'_{2m+1} \end{pmatrix} = \begin{pmatrix} \cos\theta_{t,m} & -\sin\theta_{t,m} \\ \sin\theta_{t,m} & \cos\theta_{t,m} \end{pmatrix} \begin{pmatrix} x_{2m} \\ x_{2m+1} \end{pmatrix}$$



Takeaway

- RoPE does not add a positional vector; it **rotates** query and key coordinates.
- The attention score then depends naturally on **relative position**.
- It is simple, parameter-free, and works well in modern LLMs.



$$\mathbf{q}, \mathbf{k} \mapsto \mathbf{R}_t(\mathbf{q}, \mathbf{k})$$

Section 4

Normalization

Observation: Poorly calibrated dynamic range of activations

$\Longrightarrow$ Poor backpropagation (**vanishing / exploding gradients**)

How to measure variability?

1. **Across data points** (mini-batch statistics)
   $\rightsquigarrow$ **Batch Normalization**
2. **Across features (within a layer)**
   $\rightsquigarrow$ **Layer Normalization**
3. **Across features (scale only, no centering)**
   $\rightsquigarrow$ **RMS Normalization (RMSNorm)**

RMS Normalization (RMSNorm): Idea

Goal: Normalize activations without subtracting the mean.

Given an input vector $\mathbf{x} \in \mathbb{R}^d$, RMSNorm rescales it using its root-mean-square (RMS):

$$\mathrm{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}$$

$$\mathrm{RMSNorm}(\mathbf{x}) = \gamma \cdot \frac{\mathbf{x}}{\mathrm{RMS}(\mathbf{x}) + \epsilon}$$

Key differences from LayerNorm:

- No mean subtraction
- Only rescales magnitude (not centered)
- Fewer operations $\Rightarrow$ faster

Pre-Norm vs. Post-Norm Transformers

Let $\mathbf{x}$ be the input to a Transformer sublayer F.

Post-Norm (original Transformer): normalization is applied **after** the residual addition.

$$\mathbf{x}_{\ell+1} = \mathrm{Norm}(\mathbf{x}_\ell + F(\mathbf{x}_\ell))$$

Pre-Norm: normalization is applied **before** the sublayer.

$$\mathbf{x}_{\ell+1} = \mathbf{x}_\ell + F(\mathrm{Norm}(\mathbf{x}_\ell))$$

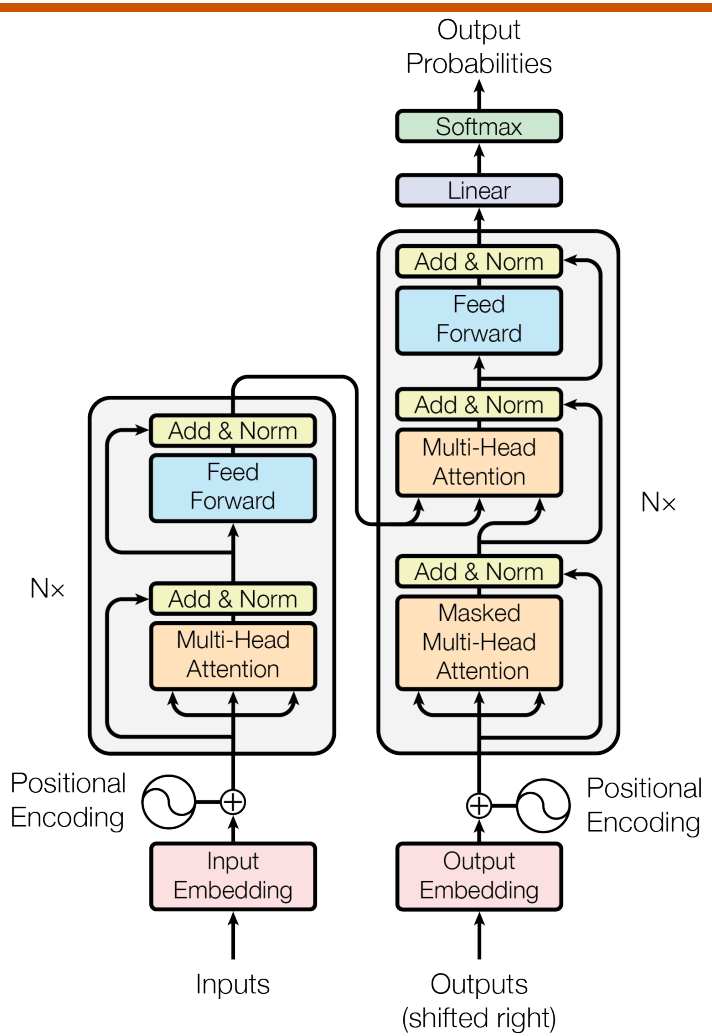
Key differences:

- Pre-Norm provides a direct residual path for gradients $\Rightarrow$ typically more stable for very deep Transformers.
- Most modern large language models use a Pre-Norm architecture.

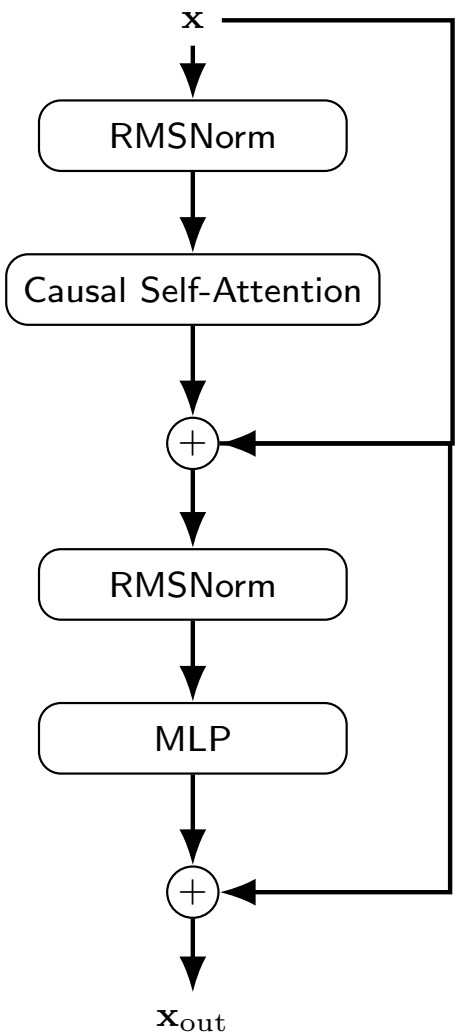
Section 5

Transformer Model

Transformer Block



Original architecture. Encoder–decoder Transformer from **Attention Is All You Need** (Vaswani et al.).



Modern decoder block. Pre-norm decoder-only block.

Transformer Block - Mathematical Formulation

A pre-norm Transformer layer can be written as

$$\widetilde{\mathbf{H}}^{\,l} = \mathbf{H}^{l-1} + \mathrm{MultiHead}\left(\mathrm{RMSNorm}\left(\mathbf{H}^{l-1}\right)\right),$$

followed by the feed-forward network:

$$\mathbf{H}^{l} = \widetilde{\mathbf{H}}^{\,l} + \mathrm{MLP}\left(\mathrm{RMSNorm}\left(\widetilde{\mathbf{H}}^{\,l}\right)\right).$$

Here,

$$\mathbf{H}^{0} = \text{input embeddings},$$

and after L Transformer layers,

$$\mathbf{H} = \mathbf{H}^{L}.$$

Transformer Block: Code

```python

class MLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd, bias=False)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd, bias=False)

    def forward(self, x):
        x = self.c_fc(x)
        x = F.relu(x).square()
        x = self.c_proj(x)
        return x


class Block(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        self.attn = CausalSelfAttention(config, layer_idx)
        self.mlp = MLP(config)

    def forward(self, x, ve, cos_sin, window_size):
        x = x + self.attn(norm(x), ve, cos_sin, window_size)
        x = x + self.mlp(norm(x))
        return x
```

ReLU square activation: weak responses are attenuated, strong responses become dominant.

Gaussian Error Linear Unit (GELU)

Goal: Smoothly gate activations based on their magnitude.

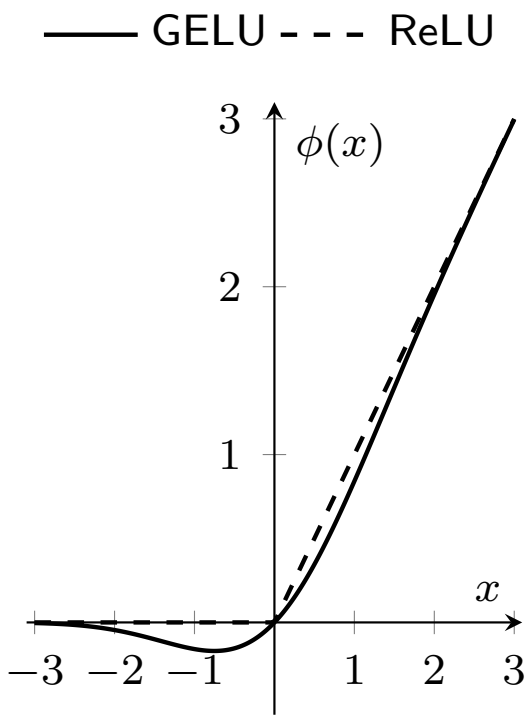
The GELU activation is defined as

$$\mathrm{GELU}(x) = x\Phi(x),$$

where $\Phi(x)$ is the CDF of the standard Gaussian distribution.

Key properties:

- Smooth transition around $x = 0$
- Negative inputs are attenuated rather than hard-zeroed
- Used in BERT, GPT-2, GPT-3, and many early Transformers



GELU provides a smooth alternative to ReLU.

Goal: Use a learned gate to control which features pass through the MLP.

SiLU activation:

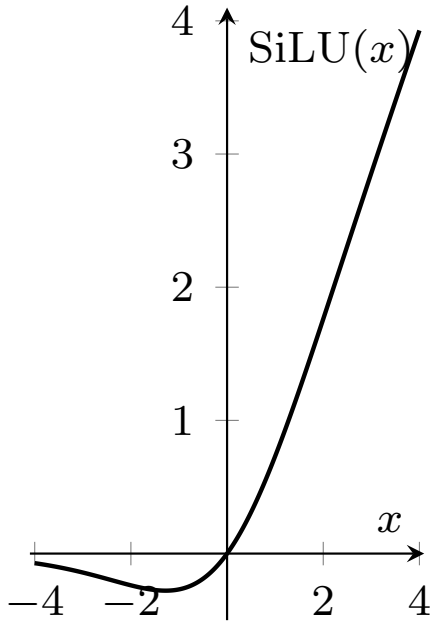
$$\mathrm{SiLU}(x) = x\sigma(x) \quad \sigma = \text{sigmoid}$$

A SwiGLU feed-forward block computes

$$\mathrm{SwiGLU}(\mathbf{x}) = \mathrm{SiLU}(\mathbf{W}_g\mathbf{x}) \odot (\mathbf{W}_u\mathbf{x}),$$

followed by a linear projection:

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_d\left[\mathrm{SiLU}(\mathbf{W}_g\mathbf{x}) \odot (\mathbf{W}_u\mathbf{x})\right].$$

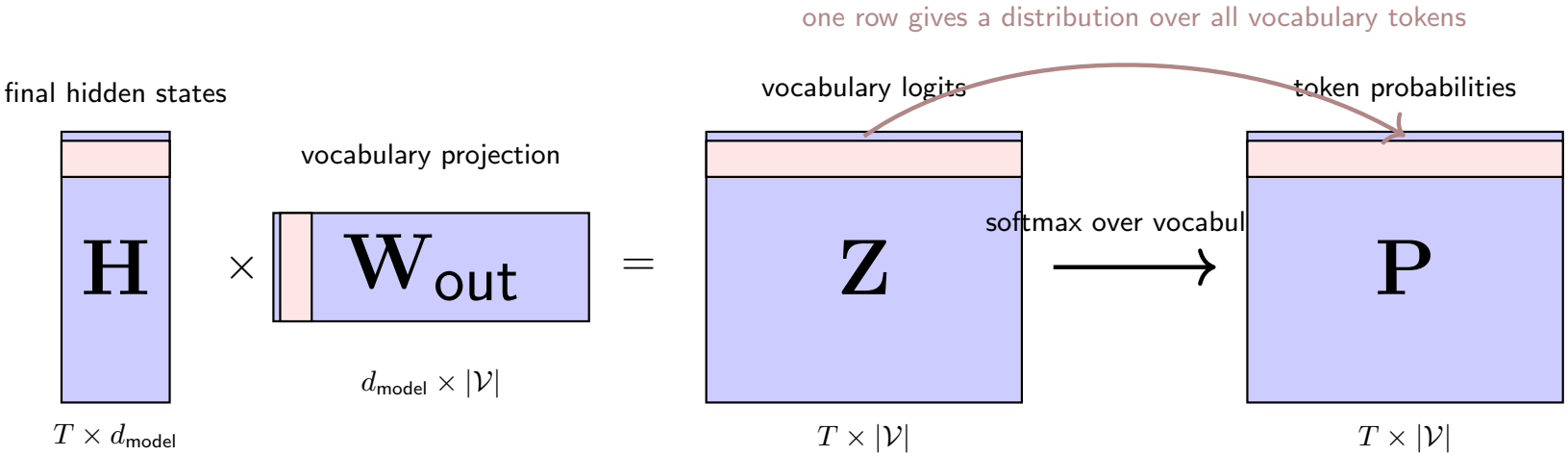


$\mathrm{SiLU}(x) = x\sigma(x)$

Key properties:

- Multiplicative gating makes the FFN more expressive
- Used in Llama, Mistral, Qwen, DeepSeek, etc.

Predictions



Section 6

Coding the Attention Mechanism

Attention Mechanism: Code (Single-head)

```python
class AttentionBlock(nn.Module):

    def __init__(self, embed_dim, hidden_dim):
        super().__init__()
        self.ln = nn.LayerNorm(embed_dim)
        self.Wq = nn.Linear(embed_dim, embed_dim)
        self.Wk = nn.Linear(embed_dim, embed_dim)
        self.Wv = nn.Linear(embed_dim, embed_dim)
        self.Wproj = nn.Linear(embed_dim, embed_dim)  # output projection
            after attention
        self.ln2 = nn.LayerNorm(embed_dim)
        self.mlp = nn.Sequential(
            nn.Linear(embed_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, embed_dim))

    # x has shape [batch_size, sequence_length, embed_dim]
    def forward(self, x):
        B, T, C = x.shape
        x_norm = self.ln(x)
        Q = self.Wq(x_norm)
        K = self.Wk(x_norm)
        V = self.Wv(x_norm)
        scores = torch.matmul(Q, K.transpose(-2, -1)) / (C ** 0.5)
        scores = scores.masked_fill(causal_mask(T, x.device), float('-inf'))
        attn = torch.matmul(torch.softmax(scores, dim=-1), V)
        attn = self.Wproj(attn)
        x = x + attn  # residual connection
        return x + self.mlp(self.ln2(x))  # residual connection
```

nanoGPT: GPT in ∼600 lines of PyTorch

- Minimal, readable codebase by Andrej Karpathy for training & fine-tuning GPT-style decoder-only Transformers
- Two files do the work:
  - `model.py`: the GPT model (∼300 lines)
  - `train.py`: the training loop (∼300 lines)
- Reproduces GPT-2 (124M) on OpenWebText
- Practical features: multi-head causal attention via FlashAttention, `torch.compile`, mixed precision, multi-GPU (DDP)
- Toy entry point: character-level Shakespeare on a laptop

Attention Mechanism: Code (nanoGPT)

```python
import torch.nn as nn

class CausalSelfAttention(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        self.n_head = config.n_head
        self.n_kv_head = config.n_kv_head
        self.n_embd = config.n_embd
        self.head_dim = self.n_embd // self.n_head
        self.c_q = nn.Linear(self.n_embd, self.n_head * self.head_dim, bias=
            False)
        self.c_k = nn.Linear(self.n_embd, self.n_kv_head * self.head_dim, bias
            =False)
        self.c_v = nn.Linear(self.n_embd, self.n_kv_head * self.head_dim, bias
            =False)
        self.c_proj = nn.Linear(self.n_embd, self.n_embd, bias=False)
        self.ve_gate_channels = 32
        self.ve_gate = nn.Linear(self.ve_gate_channels, self.n_kv_head, bias=
            False) if has_ve(layer_idx, config.n_layer) else None
```

- nn.Linear maps each input feature vector in a tensor to an output feature vector via a learned affine transformation, i.e. it computes $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$

Attention Mechanism: Code

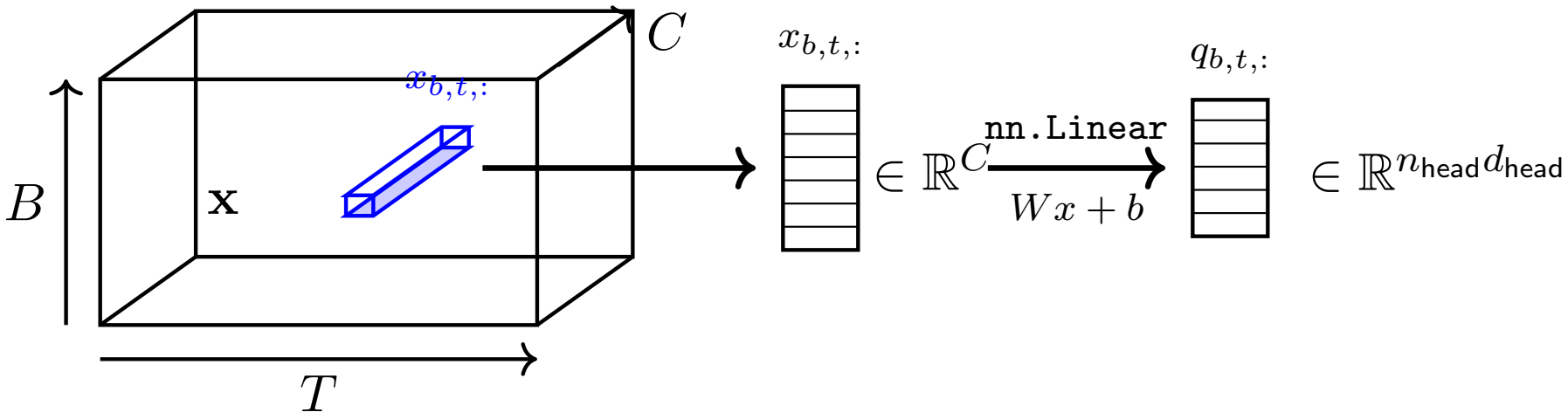


Figure: 'nn.Linear' acts on the last dimension C: for each fixed (b, t), it maps the feature vector $x_{b,t,:} \in \mathbb{R}^C$ to an output vector.

Attention Mechanism: Code

```
class CausalSelfAttention(nn.Module):

    def forward(self, x, ve, cos_sin, window_size):
        B, T, C = x.size()
        q = self.c_q(x).view(B, T, self.n_head, self.head_dim)
        k = self.c_k(x).view(B, T, self.n_kv_head, self.head_dim)
        v = self.c_v(x).view(B, T, self.n_kv_head, self.head_dim)

        ...
```

The input $\mathbf{x}$ has shape (B, T, C) where:

- B — number of sequences in the batch
- T — sequence length (number of tokens)
- C — embedding dimension (d in earlier notation)

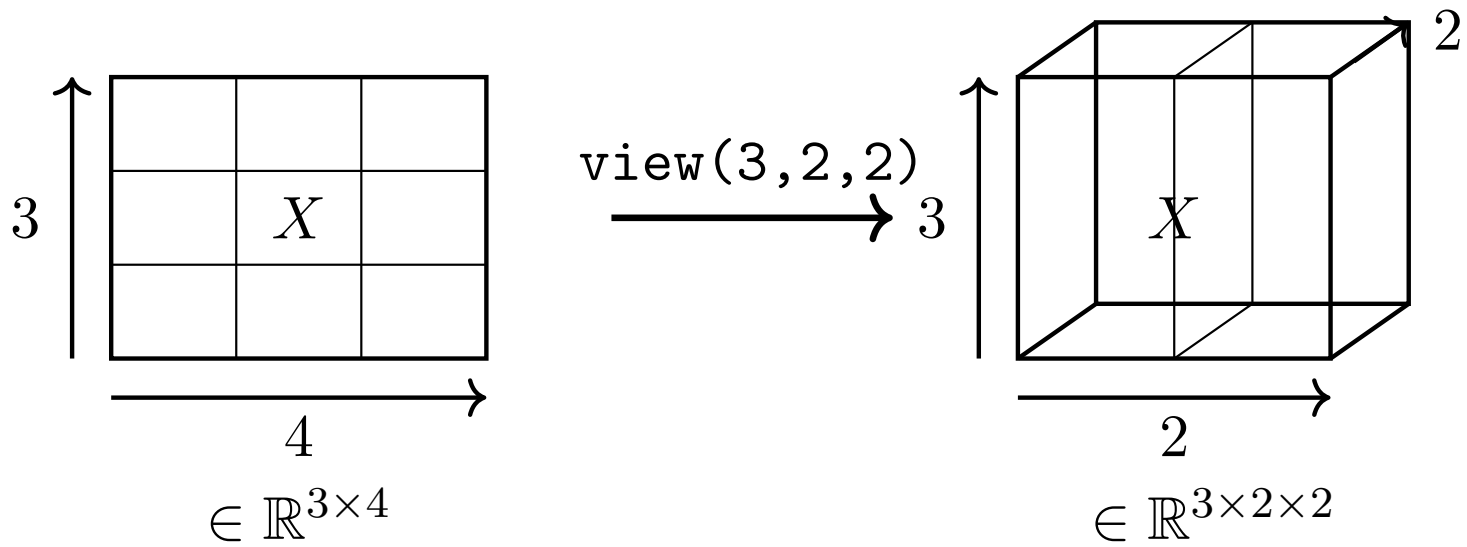


Figure: A simple example of `view`: reshaping a 3×4 tensor into a $3 \times 2 \times 2$ tensor by splitting the second dimension.

Attention Mechanism: Code

```
class CausalSelfAttention(nn.Module):

    def forward(self, x, ve, cos_sin, window_size):
        B, T, C = x.size()
        q = self.c_q(x).view(B, T, self.n_head, self.head_dim)
        k = self.c_k(x).view(B, T, self.n_kv_head, self.head_dim)
        v = self.c_v(x).view(B, T, self.n_kv_head, self.head_dim)

        ...
```

Naive implementation

$$\rightsquigarrow \text{Matrix form: } \mathbf{Y} = \text{Softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$$

```
1  def scaled_dot_product_attention(Q, K, V):
2      """
3      Q, K, V: (batch_size, seq_len, embed_dim)
4      Returns:
5        output: (batch_size, seq_len, embed_dim)
6        attn_weights: (batch_size, seq_len, seq_len)
7      """
8      d_k = Q.size(-1)  # embed_dim in single-head scenario
9
10     scores = torch.matmul(Q, K.transpose(-2, -1))/(d_k**0.5)
11     attn_weights = torch.softmax(scores, dim=-1)
12
13     output = torch.matmul(attn_weights, V)
14
15     return output, attn_weights
```

Problem: Memory and IO Bottleneck

The matrix $\mathbf{QK}^\top \in \mathbb{R}^{T \times T}$ requires:

- $\mathcal{O}(T^2)$ memory to store,
- multiple reads and writes to high-bandwidth memory,

which becomes the main bottleneck for large T.

Alternative: FlashAttention

FlashAttention computes the same quantity without materializing $\mathbf{QK}^\top$. Instead, it:

- processes queries, keys, and values in blocks,
- keeps intermediate results in fast on-chip memory,
- fuses attention computation into a single kernel.

Attention Mechanism: Code

```python
from kernels import get_kernel
cap = torch.cuda.get_device_capability()
# varunneal's FA3 is Hopper only, use kernels-community on non-Hopper GPUs
repo = "varunneal/flash-attention-3" if cap == (9, 0) else "kernels-community/flash-attn3"
fa3 = get_kernel(repo).flash_attn_interface

class CausalSelfAttention(nn.Module):

    def forward(self, x, ve, cos_sin, window_size):
        B, T, C = x.size()
        q = self.c_q(x).view(B, T, self.n_head, self.head_dim)
        k = self.c_k(x).view(B, T, self.n_kv_head, self.head_dim)
        v = self.c_v(x).view(B, T, self.n_kv_head, self.head_dim)

        cos, sin = cos_sin
        q, k = apply_rotary_emb(q, cos, sin), apply_rotary_emb(k, cos, sin)
        q, k = norm(q), norm(k)

        y = fa3.flash_attn_func(q, k, v, causal=True, window_size=window_size)
        y = y.contiguous().view(B, T, -1)
        y = self.c_proj(y)
        return y
```

Merging Heads and Output Projection

Step 1: Merge attention heads

After attention, we have:

$$\mathbf{y} \in \mathbb{R}^{B \times T \times n_{\text{head}} \times \text{head_dim}}$$

We reshape:

$$(B, T, n_{\text{head}}, \text{head_dim}) \longrightarrow (B, T, n_{\text{embd}})$$

```
y = y.contiguous().view(B, T, -1)
```

- Flattens head and channel dimensions
- `contiguous()` ensures memory layout is compatible with `view`

Step 2: Output projection

```
1 y = self.c_proj(y)
```

$$\mathbf{y} \leftarrow \mathbf{W}_O \mathbf{y}$$

- $\mathbf{W}_O \in \mathbb{R}^{n_{\text{embd}} \times n_{\text{embd}}}$
- Mixes information across heads
- Standard final step in multi-head attention

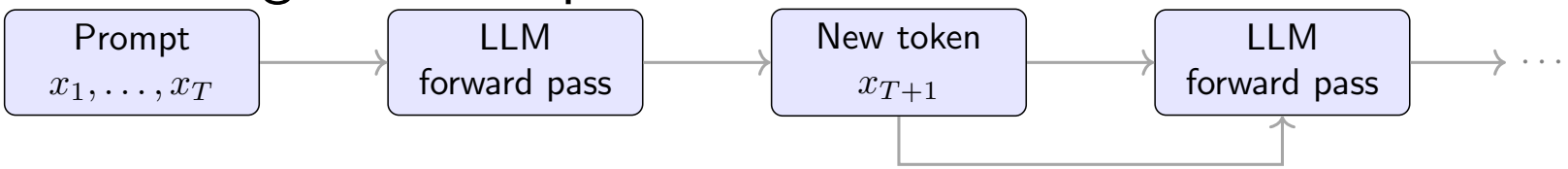
Section 7

Inference

Given a prompt (sequence of tokens), an LLM generates a response **one token at a time**:

1. **Prefill phase:** the full prompt $\mathbf{X} \in \mathbb{R}^{T \times d}$ is processed in one forward pass — all queries, keys, and values are computed simultaneously, exactly as in training
2. **Decode phase:** the model generates tokens autoregressively — at each step t, the newly generated token is fed back as input to produce the next one

The autoregressive loop



Key challenge At each decode step, the model must attend over all previous tokens — naively recomputing everything is expensive. This motivates the **KV-cache**.

Inference – Autoregressive Decoding: One Token at a Time

At decode step t, the full sequence $\mathbf{X}$ is **not available** — we build it one row at a time. The only new input is the embedding of the current token:

$$\mathbf{x}_t \in \mathbb{R}^{1\times d}$$

The three projections are computed for this single vector:

$$\mathbf{q}_t = \mathbf{x}_t\mathbf{W}^q \qquad \mathbf{k}_t = \mathbf{x}_t\mathbf{W}^k \qquad \mathbf{v}_t = \mathbf{x}_t\mathbf{W}^v$$

All three are computed, but their **fate differs**:

Cached $\mathbf{k}_t$ and $\mathbf{v}_t$

Appended to the KV-cache and **kept** for all future decode steps.

Discarded $\mathbf{q}_t$

Used **immediately** to compute attention, then **discarded** — future steps will never need it.

The attention output for the current token is:

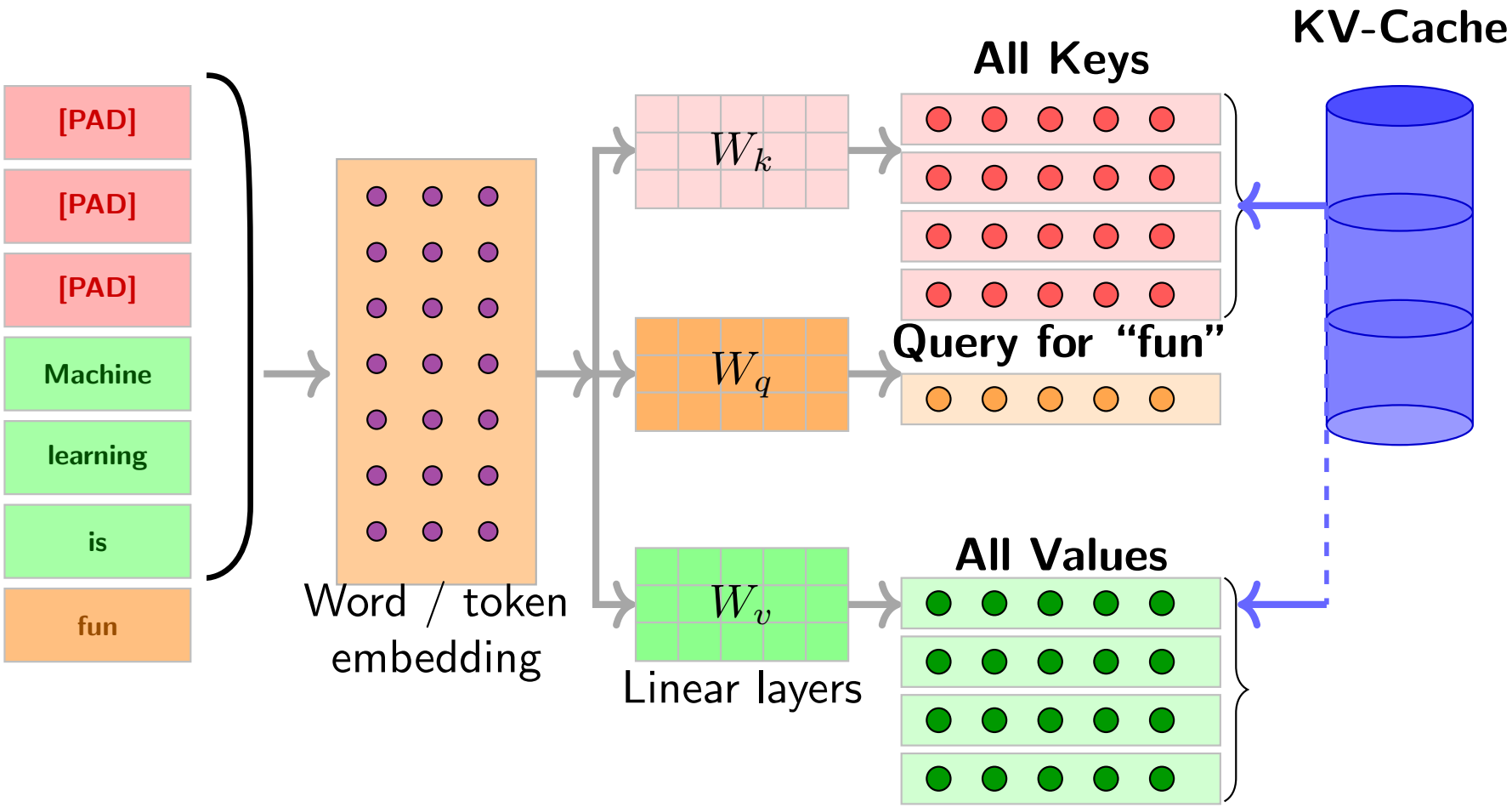
$$\mathbf{y}_t = \text{Softmax}\left(\frac{\mathbf{q}_t \, \mathbf{K}_{1:t}^\top}{\sqrt{d_k}}\right) \mathbf{V}_{1:t}$$

- $\mathbf{K}_{1:t} \in \mathbb{R}^{t \times d_k}$ and $\mathbf{V}_{1:t} \in \mathbb{R}^{t \times d_k}$ are **retrieved from cache** — all rows except the last were computed in previous steps
- Only the new row $\mathbf{k}_t, \mathbf{v}_t$ is freshly computed and then appended
- $\mathbf{q}_t \mathbf{K}_{1:t}^\top \in \mathbb{R}^{1 \times t}$ gives one attention score per past token
- $\mathbf{y}_t \in \mathbb{R}^{1 \times d_k}$ is a weighted sum of all cached value vectors

Efficiency gain

Without the cache, all keys and values would be recomputed from scratch at every step: $\mathcal{O}(T^2)$ cost. With the cache, each step appends one row: $\mathcal{O}(T)$ incremental compute.

KV Cache


[PAD]
[PAD]
[PAD]
Machine
learning
is
fun
Word / token embedding
W_k
W_q
W_v
Linear layers
All Keys
Query for "fun"
All Values
KV-Cache

Section 8

Training

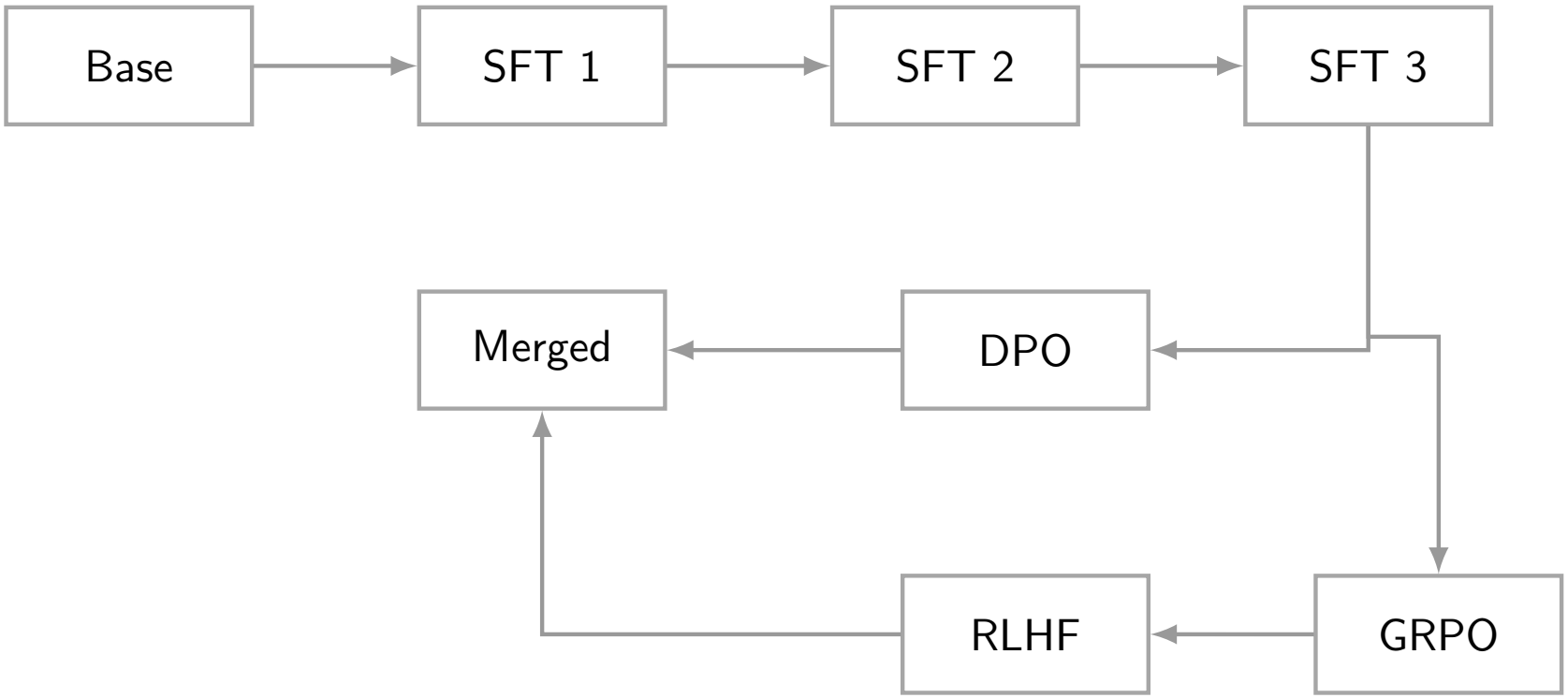
Section 8.1

Training Phases

Training Phases

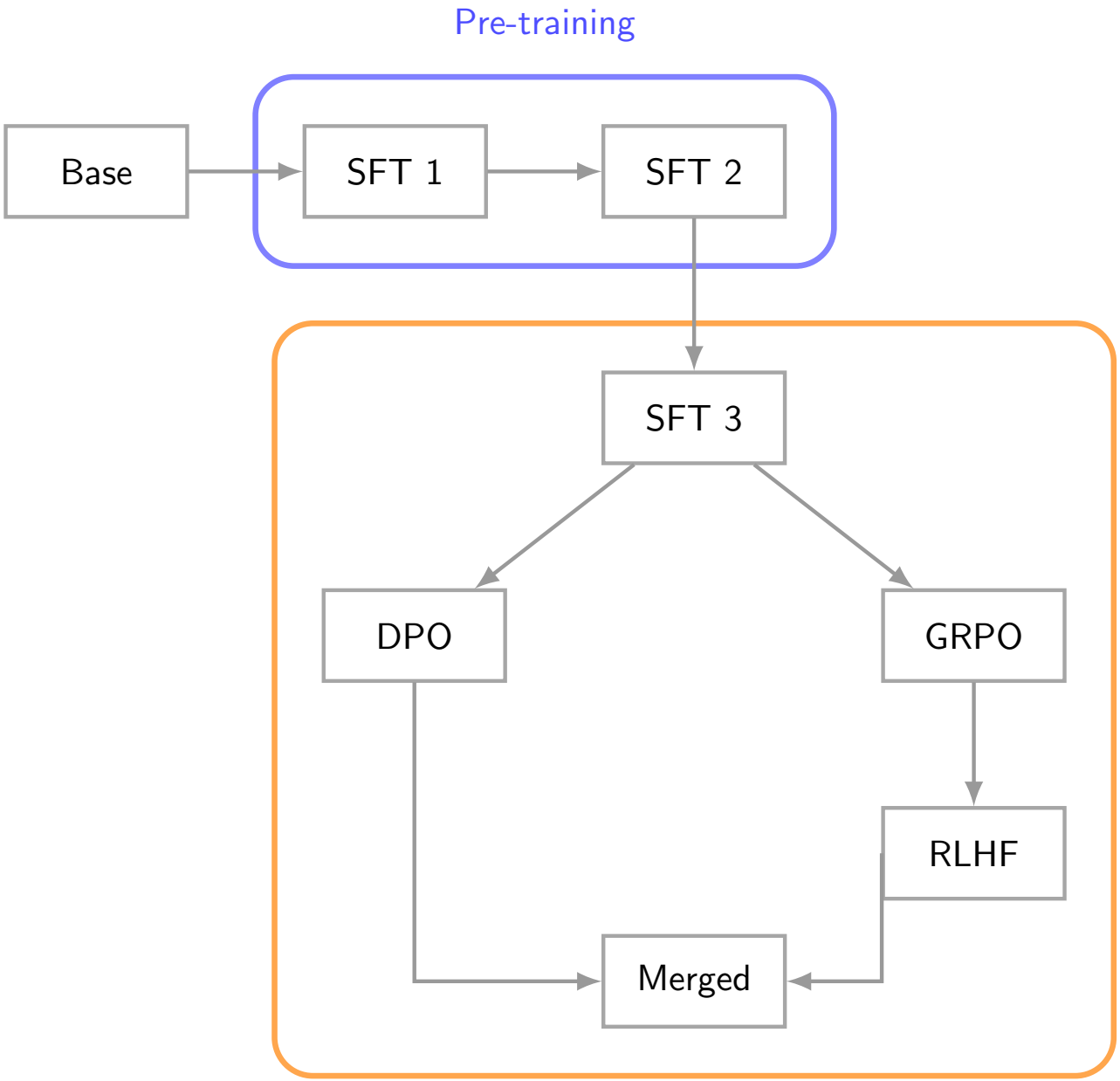
- **Pre-training:** curriculum based on a three-phase data blending strategy
  - First phase employs a diverse data mixture to maximize coverage and robustness. In the second and third phases, shift towards higher-quality data sources (e.g., Wikipedia), progressively refining the model's knowledge and improving signal-to-noise ratio.
- **Post-training:** multi-stage supervised fine-tuning (SFT) pipeline
  - Apply successive SFT phases targeting different capabilities, including general instruction following, domain-specific skills (e.g., reasoning, tool use, long-context understanding), and fine-grained behavior control. This staged approach mitigates interference between objectives and enables more precise alignment.

Training Phases



Source: [Basant et al., 2025]

Training Phases



SFT = supervised learning, usually cross-entropy on the target answer.

Why SFT Is Not Enough?

- SFT teaches imitation of one reference answer.
- But many prompts have multiple valid answers.
- Cross-entropy does not directly teach the model to prefer better answers over worse alternatives.
- Preference optimization helps when quality is relative.
- Preference data can be collected from the model's own outputs, targeting its current mistakes.

Summary: SFT teaches imitation; preference optimization teaches choice.

RLHF = Reinforcement Learning from Human Feedback

SFT model $\rightarrow$ preference data $\rightarrow$ reward model $\rightarrow$ RL optimization

- Humans compare model outputs:

$$(\text{prompt},\ \text{chosen response},\ \text{rejected response})$$

- A reward model is trained to score preferred responses higher.
- The language model is then optimized to produce responses with higher reward.
- A KL penalty is usually added so the model does not drift too far from the SFT model.

$$\max_{\pi}\ \mathbb{E}_{y\sim\pi(\cdot|x)}\left[r(x,y) - \beta\,\mathrm{KL}(\pi||\pi_{\mathsf{SFT}})\right]$$

PPO vs. GRPO in RLHF

PPO: classical RLHF optimizer

- Uses a reward model to score generated responses.
- Trains a policy model to increase reward.
- Trains a value function / critic to estimate expected reward.
- Uses clipping and KL regularization for stable updates.

GRPO: group-relative optimization

- Samples multiple responses for the same prompt.
- Compares each response to the group average reward.
- Avoids training a separate value function / critic.

RLHF = training paradigm PPO/GRPO = optimization algorithms

GRPO: Group Relative Policy Optimization

Goal. Improve a language model with reinforcement learning using a reward signal r (e.g. correctness, format, preference), while avoiding the extra cost of training a separate value model as in PPO.

Main idea. For each prompt q, sample a group of completions

$$o_1, \ldots, o_G \sim \pi_{\theta_{\text{old}}}(\cdot \mid q),$$

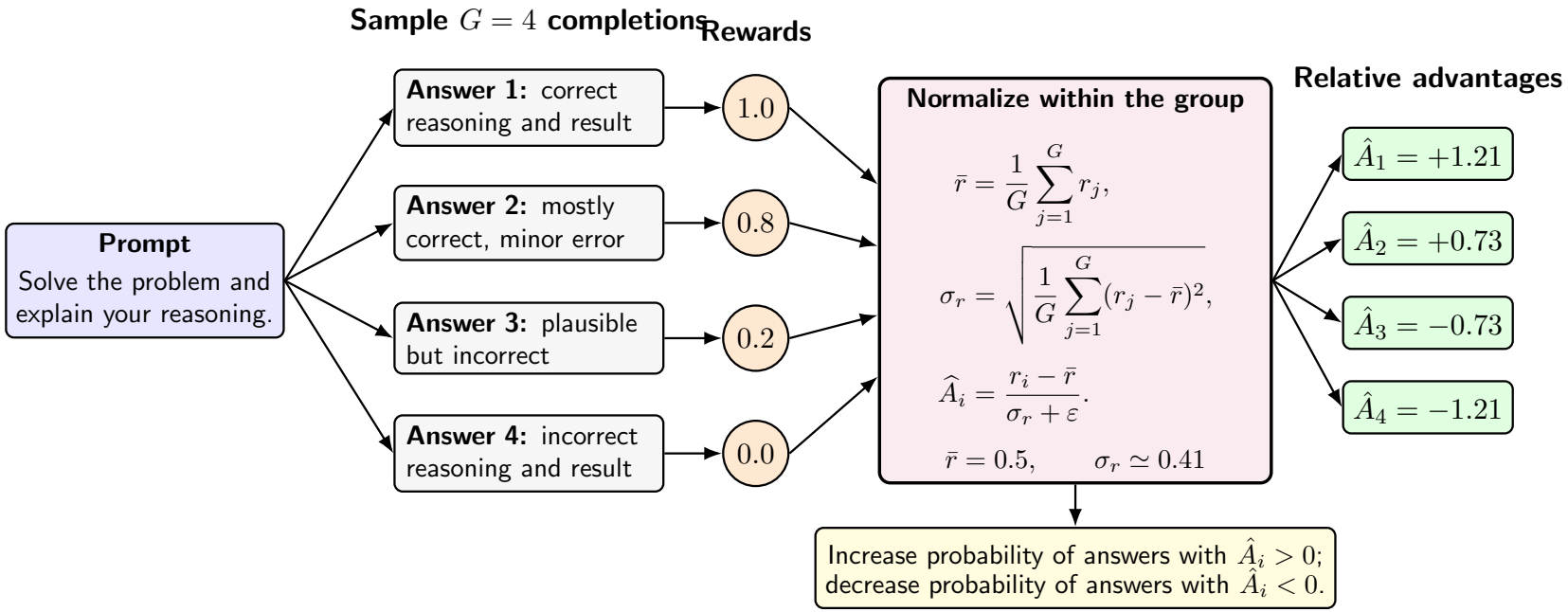
score them with rewards $r_1, \ldots, r_G$, and compare each completion only **relative to the others in the same group**.

Group-normalized advantage.

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}.$$

GRPO was introduced in DeepSeekMath as a critic-free alternative to PPO.

GRPO: learning from relative rewards



GRPO objective and intuition

For prompt q and sampled completion o_i, define the importance ratio

$$\rho_i = \frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}.$$

A PPO-style GRPO objective is

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} \min\Big(\rho_i A_i,\ \text{clip}(\rho_i, 1-\varepsilon, 1+\varepsilon)A_i\Big)\right].$$

Interpretation.

- If completion o_i gets above-average reward, then $A_i > 0$: increase its probability.
- If it gets below-average reward, then $A_i < 0$: decrease its probability.
- Clipping with ε prevents overly large policy updates.

This group-relative baseline is the key distinction between GRPO and standard PPO.

Section 8.2

Training Data

Supervised Fine-Tuning

$$\text{prompt} \rightarrow \text{ideal response}$$

SFT learns from instruction-response pairs.

Reward Modeling

$$(\text{prompt},\ \text{chosen response},\ \text{rejected response})$$

The reward model learns to assign:

$$r(\text{prompt}, \text{chosen}) > r(\text{prompt}, \text{rejected})$$

FineWeb Dataset

FineWeb is a large-scale web dataset for language model pretraining.

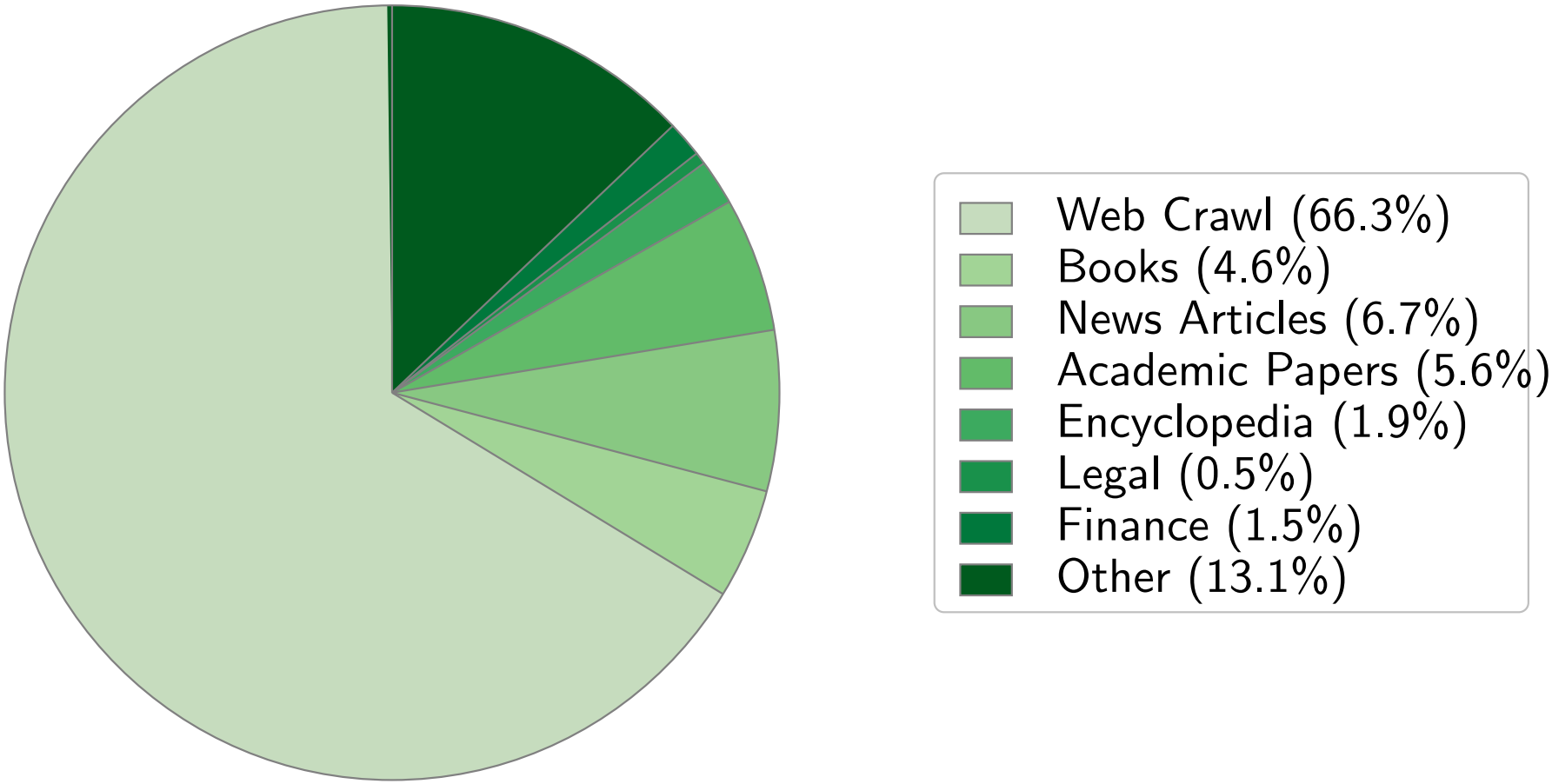
- Built from large web crawls and heavily filtered for quality
- Designed for **LLM pretraining** at scale
- Includes steps such as:
  - text extraction and language filtering
  - removal of low-quality / noisy pages
  - deduplication to reduce repeated content
- Goal: improve the quality of training tokens compared to unprocessed web data

Why it matters:

- High-quality pretraining data is crucial for strong language models
- Better filtering often leads to better performance per token

Pretraining Data Composition

Source: [Parmar et al., 2024]



Nemotron Nano 2: Pretraining Data Overview

Two main sources of pretraining data:

- **Curated data**
  - English web crawl data
  - Additional `CC-NEWS` data to improve the knowledge cutoff
  - Multilingual web data for 15 languages, plus Wikipedia and FineWeb-2
  - Curated math corpus extracted from Common Crawl with special care to preserve equations
  - Curated GitHub source code with license filtering, exact/fuzzy deduplication, and quality filtering
- **Synthetically-generated data**
  - STEM question/solution data
  - Synthetic math dialogue data
  - Multilingual QA data
  - Code QA data
  - Academic QA data from textbooks and papers

Goal: combine broad coverage from curated corpora with high-value reasoning-oriented synthetic data.

Section 9

Interpretability

Logit Lens: Decoding Intermediate Representations

Consider an input sequence of T tokens $x_1, \ldots, x_T$.

After Transformer layer ℓ, the model produces the hidden-state matrix

$$\mathbf{H}^{(\ell)} = \begin{bmatrix} (\mathbf{h}_1^{(\ell)})^\top \\ \ldots \\ (\mathbf{h}_T^{(\ell)})^\top \end{bmatrix} \in \mathbb{R}^{T \times d},$$

where $\mathbf{h}_t^{(\ell)} \in \mathbb{R}^d$ is the representation at position t.

The final language-model head is a linear map

$$\mathrm{LMHead}(\mathbf{h}) = \mathbf{W}_U \mathbf{h} + \mathbf{b}_U \in \mathbb{R}^{|\mathcal{V}|}, \qquad \mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d}.$$

$\rightsquigarrow$ Each output coordinate is the logit assigned to one possible next token in the vocabulary.

Logit Lens: Decoding Intermediate Representations

The **logit lens** [nostalgebraist, 2020] applies this same output map to every intermediate layer:

$$\mathbf{z}_t^{(\ell)} = \mathbf{W}_U \mathbf{h}_t^{(\ell)} + \mathbf{b}_U \in \mathbb{R}^{|\mathcal{V}|}.$$

Thus, $\mathbf{z}_t^{(\ell)}$ assigns a score to every token in the vocabulary $\mathcal{V}$.

The intermediate logits are converted into a probability distribution:

$$p_t^{(\ell)}(v) = \frac{\exp\bigl(z_{t,v}^{(\ell)}\bigr)}{\sum_{u \in \mathcal{V}} \exp\bigl(z_{t,u}^{(\ell)}\bigr)}.$$

We then choose the most likely token as:

$$\widehat{x}_{t+1}^{(\ell)} = \arg\max_{v \in \mathcal{V}} z_{t,v}^{(\ell)}.$$

Causal interpretation

The hidden state at position t contains information from $x_1, \ldots, x_t$. Therefore, its logits predict the **next token** x_{t+1}, not the input token x_t itself.

For example, after processing

$$\texttt{The capital of France is},$$

the representation at the last position may progressively predict

$$\texttt{city} \longrightarrow \texttt{Paris}$$

as the layer index increases.

Entropy Heatmap and Implementation Details

We can measure the uncertainty of each intermediate prediction using the entropy

$$\mathcal{H}_t^{(\ell)} = -\sum_{v \in \mathcal{V}} p_t^{(\ell)}(v) \log p_t^{(\ell)}(v).$$

- Low entropy: the model strongly favors a small number of tokens.
- High entropy: the probability mass is spread across many tokens.

Example

Prompt = "The capital of France is"

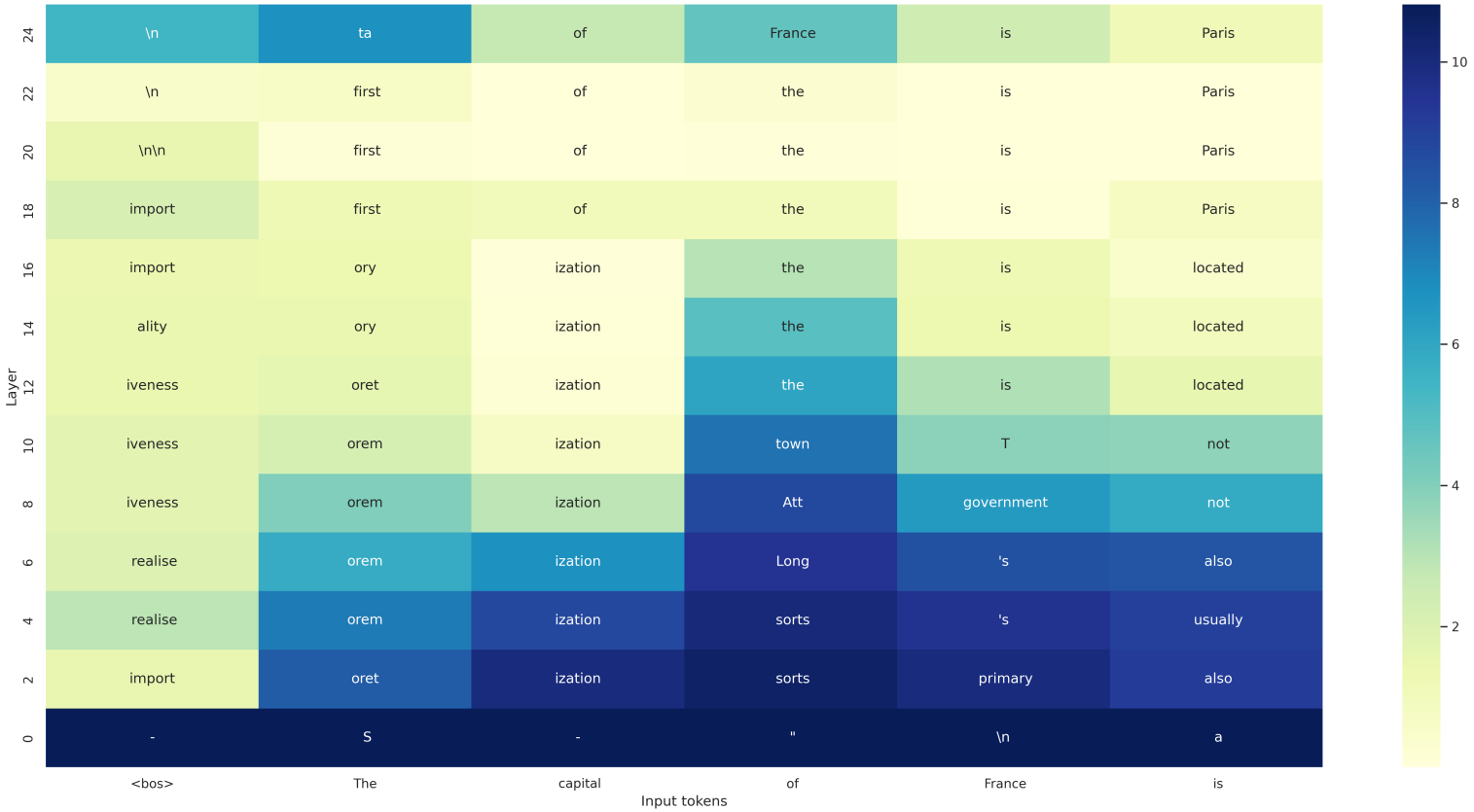


Figure: Logit lens output with microsoft/phi-1.5 model. Darker blue cells indicate higher entropy and therefore greater uncertainty in the model's token prediction, while lighter yellow-green cells indicate lower entropy and greater confidence.

Example

Prompt = "The capital of France is"

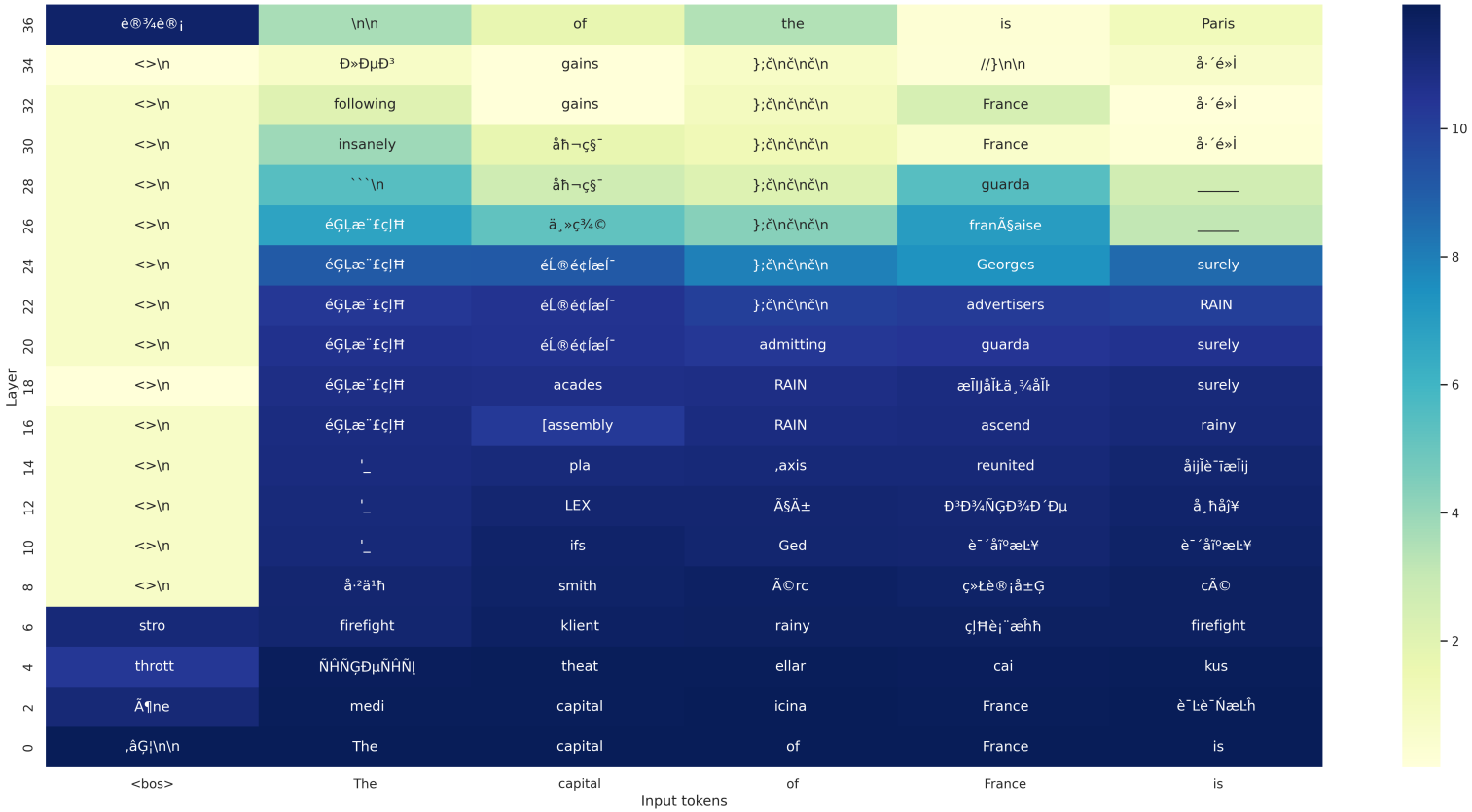


Figure: Logit lens output with Qwen3-4B.

Example

Prompt = "Albert Einstein developed the theory of"

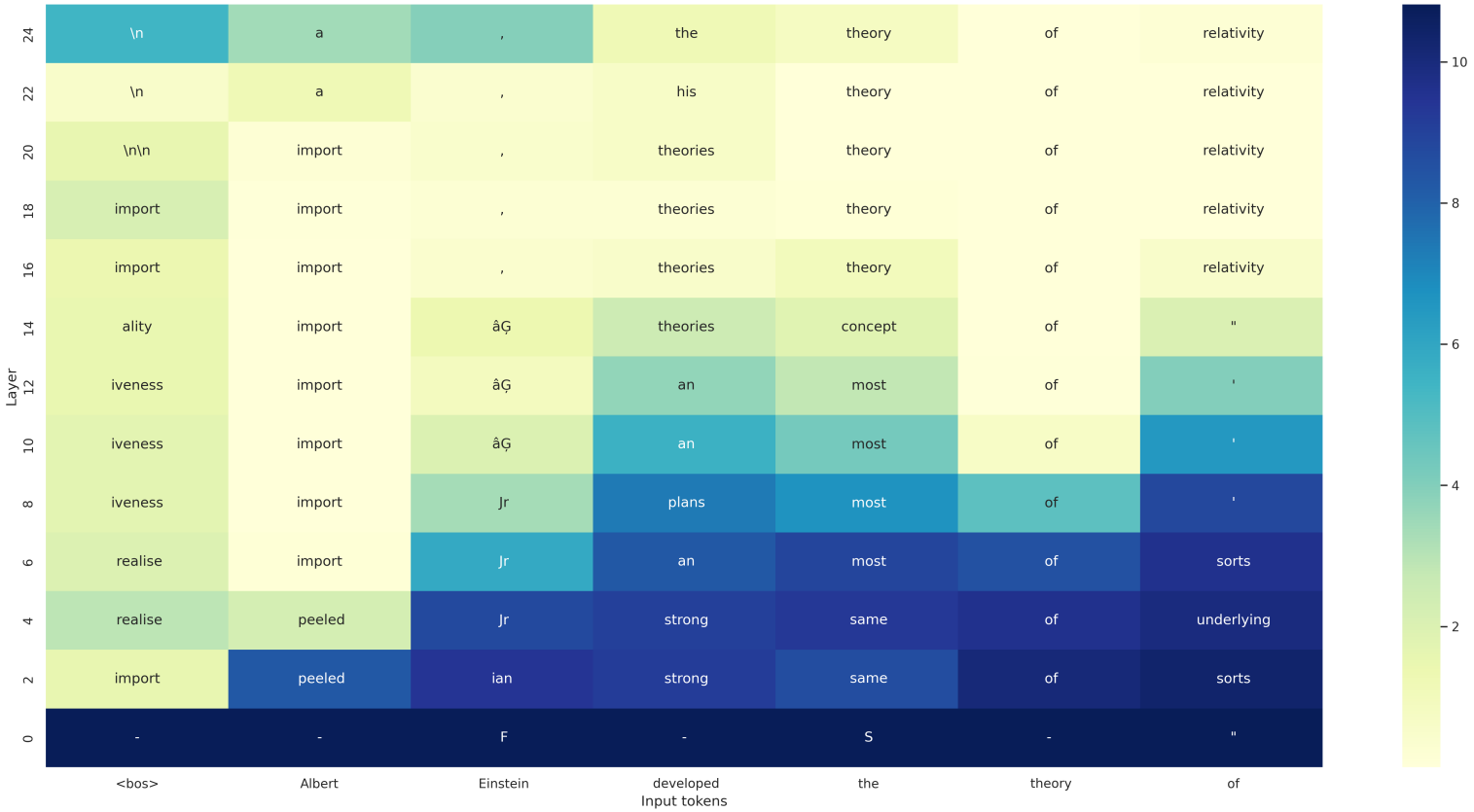


Figure: Logit lens output with microsoft/phi-1.5 model.

Example

Prompt = "Albert Einstein developed the theory of"

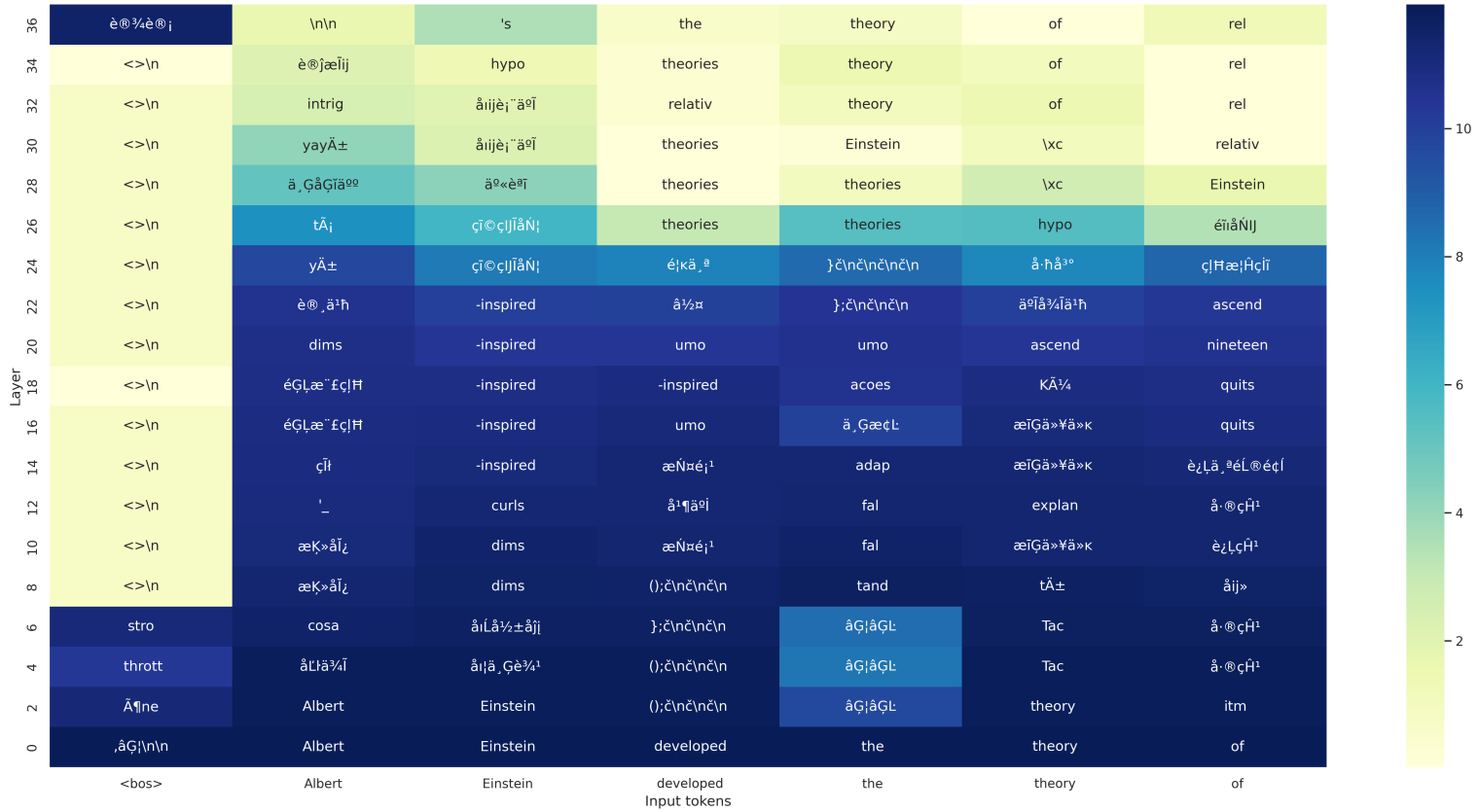


Figure: Logit lens output with Qwen3-4B.

References



Basant, A., Khairnar, A., Paithankar, A., Khattar, A., Renduchintala, A., Malte, A., Bercovich, A., Hazare, A., Rico, A., Ficek, A., et al. (2025).

Nvidia nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model.

arXiv preprint arXiv:2508.14444.



nostalgebraist (2020).

Interpreting GPT: The logit lens.

LessWrong.



Parmar, J., Prabhumoye, S., Jennings, J., Patwary, M., Subramanian, S., Su, D., Zhu, C., Narayanan, D., Jhunjhunwala, A., Dattagupta, A., et al. (2024).

Nemotron-4 15b technical report.

arXiv preprint arXiv:2402.16819.